Silesian
University
of Technology

MASTER THESIS

Granular Computations for Sentiment Detection in Natural Language Texts

Jan SUPIERZ
Student identification number: 319212

Programme: Informatics
Specialisation: System Software

SUPERVISOR
dr hab. inż. Krzysztof Simiński PhD DSc, prof. PŚ
DEPARTMENT of Algorithmics and Software
Faculty of Automatic Control, Electronics and Computer Science

Gliwice 2026

Thesis title

Granular Computations for Sentiment Detection in Natural Language Texts

Abstract

Sentiment analysis models face a persistent trade-off between the high accuracy of deep learning and the computational efficiency required for practical use. This thesis investigates whether granular computing techniques can bridge this gap. A statistical Z-score pruning method is proposed that discards non-discriminative n-grams, shrinking the feature space by over 60% while maintaining or improving classification accuracy (RQ1). Semantic concept clustering is explored as an alternative granulation strategy. Building on the best lightweight model, a cascade ensemble architecture is proposed: a linear support vector machine handles the majority of reviews with high confidence, and a specialist transformer model resolves only the uncertain cases (RQ2). All methods are evaluated on the IMDb movie review dataset. The cascade achieves a macro F_1-score of approximately 92.3%, significantly outperforming both standalone baselines while delegating only 15%-20% of the test instances to the deep-learning specialist. These results demonstrate that granular vocabulary reduction and model-level deferral can successfully balance accuracy and efficiency in sentiment analysis systems.

Key words

Sentiment Analysis, Granular Computing, Cascade Ensemble, Machine Learning, Natural Language Processing

Tytuł pracy

Wykorzystanie obliczeń ziarnistych do wykrywania ładunku emocjonalnego w tekstach w języku naturalnym

Streszczenie

Modele służące do wykrywania ładunku emocjonalnego stale balansują między wysoką dokładnością uczenia głębokiego a wydajnością obliczeniową niezbędną w praktycznych zastosowaniach. Niniejsza praca bada, czy techniki obliczeń ziarnistych mogą wypełnić tę lukę. Zaproponowano statystyczną metodę odrzucania opartą na Z-score, która odrzuca niedyskryminujące n-gramy, zmniejszając przestrzeń cech o ponad 60% przy jednoczesnym utrzymaniu lub poprawie dokładności klasyfikacji (RQ1). Jako alternatywną strategię granulacji analizowane jest grupowanie pojęć semantycznych. Opierając się na najlepszym lekkim modelu, zaproponowano architekturę zespołu kaskadowego: liniowa maszyna wektorów wspierających obsługuje większość recenzji z wysoką pewnością, a wyspecjalizowany model architektury transformer rozwiązuje tylko niepewne przypadki (RQ2). Wszystkie metody oceniono na zbiorze danych recenzji filmów IMDb. Zespół kaskadowy osiąga makro-F_1 na poziomie około 92,3%, znacząco przewyższając oba samodzielne modele bazowe, przekazując przy tym jedynie 15%-20% instancji testowych do specjalisty wykorzystującego uczenie głębokie. Wyniki te pokazują, że ziarnista redukcja słownictwa oraz odraczanie decyzji na poziomie modelu mogą skutecznie równoważyć dokładność i wydajność w systemach wykrywania ładunku emocjonalnego.

Słowa kluczowe

wykrywanie ładunku emocjonalnego, obliczenia ziarniste, zespół kaskadowy, uczenie maszynowe, przetwarzanie języka naturalnego

Contents

Chapter 1

Introduction

1.1 Sentiment Analysis

This work explores the Sentiment Analysis (SA) task. The main goal is determining whether opinions expressed in text are positive or negative. SA has many diverse applications, ranging from gathering information from movie reviews—which will be explored in this work—to fields such as political analysis, where it can determine whether a candidate is liked by the public based on posted tweets.

1.2 Problem Statement

Despite the advances in SA, a tension persists between accuracy and computational efficiency. Deep learning models achieve state-of-the-art performance, but require more computational resources, limiting their deployment in resource-constrained or real-time environments. On the other hand, traditional machine learning models are efficient, but often fail to capture the nuanced, context-dependent sentiment expressed through sarcasm, negation, and implicit opinions; unless the feature space explodes. These trade-offs raise the following research questions:

RQ1: *Can granular approaches be used to optimize the sentiment analysis pipeline?*

RQ2: *Can a hybrid system be designed to combine the efficiency of lightweight classifiers with the semantic understanding of transformer models?*

1.3 Scope

This thesis explores the application of machine learning techniques, deep learning models, and a cascade system architecture to the task of binary sentiment analysis on the

Internet Movie Database (IMDb) movie review dataset. The investigation evaluates the impact of preprocessing choices, (granular) feature engineering strategies, and classifier selection. The primary aim is to determine best practices for building accurate yet efficient sentiment analysis systems, with a particular focus on the final cascade ensemble.

1.4 Thesis Outline

Chapter 1 (Introduction) provides the motivation for the work and defines the research problem and scope. **Chapter 2 (State of the Art)** surveys the existing literature on sentiment analysis, covering its levels, tasks, challenges, and solution methods (lexicon-based, classic machine learning, and deep learning), followed by an introduction to granular computing, three-way decisions, and prototype theory, and concludes with a gap analysis that identifies two specific research gaps this thesis addresses. **Chapter 3 (Granular Cascade System for Sentiment Analysis using Enumerations)** describes the proposed pipeline in detail, including data preprocessing, custom vectorization, statistical pruning, semantic clustering, and the cascade ensemble architecture. **Chapter 4 (Experiments)** presents the experimental methodology, evaluation metrics, and results across each stage of the pipeline, culminating in an evaluation of the full cascade system. **Chapter 5 (Summary)** summarises the findings, discusses limitations, and suggests directions for future work.

1.5 Author's Contribution

The contributions of this thesis are twofold:

First, an evaluation of Granular Computing (GrC) methods for SA is performed. Statistical Z-score pruning of n-gram features is shown to reduce the feature space while preserving or even improving classification accuracy. In contrast, semantic clustering into concept granules produces interpretable, but accuracy-limited representations; even when augmented with sentiment lexicon scores.

Second, a cascade ensemble architecture is proposed that uses three-way decisions paradigm at the model level. A lightweight base model handles confident predictions, and a specialist resolves uncertain cases. On the IMDb dataset, this cascade achieves a macro F_1-score of approximately 92.3%, outperforming standalone baselines, while delegating only 15-20% of instances to the deep-learning model.

Chapter 2

State of the Art

2.1 Sentiment Analysis

Sentiment analysis (or opinion mining) is an umbrella term for the computational study of people's opinions, sentiments, emotions, and attitudes expressed in written language [24]. The term includes field names and slightly different tasks associated with them: sentiment analysis, opinion mining, opinion extraction, sentiment mining, subjectivity analysis, affect analysis, emotion analysis, review mining, etc. [24] However, the main terms in academia remain both: sentiment analysis and opinion mining [24].

A foundational formalization was proposed by [24], who recognized that opinions are not monolithic but rather composed of multiple interacting elements. This insight led to defining an opinion as a quintuple:

$$(e_i, a_{ij}, s_{ijkl}, h_k, t_l), \tag{2.1}$$

where:

- e_i is the name of an i-th entity (e.g., a product, service, organization, or individual),
- a_{ij} is an j-th aspect of e_i (a specific feature or attribute of that entity),
- h_k is the k-th opinion holder (the person or organization expressing the opinion),
- t_l is the l-th instance when the opinion is expressed, and
- s_{ijkl} is the sentiment expressed toward aspect a_{ij} of entity e_i at t_l by h_k.

Following this definition, the main objective of sentiment analysis is to determine all quintuples present in given texts [1].

2.1.1 Sentiment Analysis Levels

Sentiment analysis tasks are typically categorized by their granularity of analysis, reflecting different levels at which opinions can be extracted and aggregated [1]. The choice of granularity determines both the complexity of the task and the type of insights that can be obtained. Three main levels of analysis are recognized in the literature:

- **Document-Level Sentiment Analysis (DLSA)** operates at the coarsest granularity, aiming to determine the overall opinion expressed in an entire document [4]. Following the opinion quintuple formulation, DLSA identifies the overall sentiment of the opinion holder about the entity [1]. Documents are typically classified as positive, negative, or neutral, under the assumption that the document focuses on a single entity or topic, and that the opinion is coherent [39]. The task commonly follows a two-step approach: Topic relevance retrieval followed by opinion finding [1, 27]. Considered the simplest sentiment analysis task, DLSA has been the most frequently studied granularity in the field, with 73 out of 159 reviewed articles operating at this level [1, 37].

- **Sentence-Level Sentiment Analysis (SLSA)** operates at a finer granularity by considering individual sentences as the unit of analysis [4]. This level typically involves a two-step process. The first step, known as subjectivity classification, distinguishes subjective sentences (expressing opinions) from objective sentences (presenting facts) [1, 7]. The second step, sentence sentiment classification, then assigns polarity labels (positive, negative, or neutral) to subjective sentences [1]. This approach is particularly useful when documents contain a mixture of factual statements and opinions [39]. However, a known limitation is that objective sentences can sometimes carry implicit sentiment [4]. Furthermore, sentence-level analysis commonly assumes that each sentence expresses a single sentiment; sentences containing multiple opposing sentiments require more nuanced treatment [1].

- **Aspect-Based Sentiment Analysis (ABSA)** represents the finest granularity, focusing on identifying sentiments expressed towards specific aspects or attributes of an entity [47]. Rather than assigning a single polarity to an entire document or sentence, ABSA extracts both the target of opinion and its associated sentiment, corresponding to the first three components of the opinion quintuple: entity, aspect, and sentiment [4, 1]. For example, in a restaurant review, the sentiment expressed towards *"food"* may differ from that towards *"service"* [39]. This fine-grained analysis addresses the need to know exactly what people liked or disliked [1]. Aspects may be expressed either explicitly or implicitly [1], and ABSA consequently requires two core subtasks: aspect extraction and aspect-level sentiment classification [39].

2.1.2 Sentiment Analysis Tasks

The explosive growth of user-generated content on social media, review platforms, and forums has made automated sentiment detection an essential tool for businesses, governments, and researchers [39]. Applications span brand monitoring, customer feedback analysis, political trend prediction, and mental health assessment [39]. More broadly, the field supports business intelligence, recommendation systems, review summarization, and government intelligence, demonstrating its impact [4]. As recent comprehensive surveys discuss [39], the field has evolved beyond simple polarity detection to address multiple complementary problems. Below are some of the recent fields of research:

- **Emotion Detection (ED)** extends polarity classification by identifying fine-grained emotional states such as joy, anger, sadness, or frustration [39]. Unlike binary or ternary sentiment classification, emotion detection requires more expressive representations and often relies on specialized lexical resources or machine learning algorithms [39].

- **Multi-domain sentiment classification** addresses the challenge of transferring models across different application domains [39]. For instance, a model trained on movie reviews may perform poorly on product reviews due to differences in vocabulary and linguistic patterns. Domain adaptation and transfer learning techniques are commonly employed to bridge this gap [39].

- **Multilingual sentiment analysis** aims to analyse sentiment across multiple languages [39]. This task typically leverages language-specific resources such as sentiment lexicons, large translated corpora, or language detection systems as preprocessing steps [39].

- **Multimodal Sentiment Analysis (MMSA)** integrates information from multiple channels, including text, audio, and visual signals [39]. By moving beyond purely textual analysis, MMSA enables more advanced applications such as analysing spoken reviews and vlogs, interpreting images and social media tags, and understanding human-machine interactions [39].

- **Opinion Spam Detection (OSD)** focuses on identifying deceptive or fabricated opinions intended to mislead users or automated sentiment analysis tools [39]. These deceptive opinions include fake reviews designed to manipulate public perception and social media posts that artificially promote specific individuals, products, or services [39].

2.1.3 Sentiment Analysis Challenges

Despite its importance and wide-ranging applications, sentiment analysis faces several fundamental challenges that complicate accurate opinion mining. Opinion words are necessary but not sufficient for reliable sentiment detection, and multiple linguistic and contextual factors must be addressed [4].

- **Contextual polarity ambiguity** is the core difficulty: the same sentiment-bearing word can have different polarities depending on the context in which it appears [4]. A word that is positive in one domain may be neutral or even negative in another.
- **Sarcasm and irony detection** presents particularly difficult challenges, as the literal meaning of an utterance contradicts its intended sentiment [4, 19]. When someone says something positive but actually means something negative, or vice versa, purely lexical approaches inevitably fail [4]. Recent research has emphasized the need for improved models capable of classifying sarcasm and irony [19].
- **Negation handling** is critical because the presence of negation words can reverse the polarity of an opinion [4, 19].
- **Implicit and objective opinions** are not always straightforward. Many sentences containing opinion words may not actually express any opinion, while conversely, many sentences without explicit opinion words can convey strong sentiments [4]. These are often objective sentences that present factual information yet carry implicit evaluative meaning [4].
- **Vocabulary challenges** include rapid vocabulary evolution, including new words, slang, and abbreviations that constantly emerge—particularly on social media [39]. Any practical system must therefore be robust to such linguistic shifts [39]. Broader challenges identified in the literature include algorithm optimization, language model limitations, subjectivity detection, tone-setting polarities, and data inconsistency [39]. Multilingual analysis adds further complexity, requiring either language-specific resources or large translated corpora [39].

2.2 Sentiment Analysis Methods

Sentiment analysis methods can be broadly divided into two main categories: traditional approaches and machine learning approaches [39]. Within machine learning, a further distinction exists between classic supervised approaches and modern deep learning techniques [4]. A detailed explanation of each method can be found in the following sections.

2.2.1 Lexicon-Based Methods

Lexicon-based approaches represent an alternative to supervised machine learning, operating without requiring labelled training data [1]. These methods rely on pre-compiled dictionaries of words annotated with sentiment scores (e.g., positive, negative, neutral) and are often considered a form of unsupervised sentiment analysis [1]. The core assumption is that the collective polarity of a sentence or document can be derived from the polarities of its individual words and phrases [1].

Dictionary-Based versus Corpus-Based Approaches

Lexicon-based methods are divided into two main categories: dictionary-based and corpus-based approaches [1, 30, 37].

- **Dictionary-based approaches** utilise existing lexical databases containing opinion words along with their sentiment strengths [1]. Popular sentiment lexicons in this category include SentiWordNet [2], WordNet-Affect [40], and VADER [18] as stated in [30]. The dictionary-based method is generally more practical as it does not require a comprehensive corpus covering all possible words [30].

- **Corpus-based approaches**, by contrast, rely on the probability of sentiment words co-occurring with positive or negative seed words within large text collections [1]. This method is particularly useful for discovering new sentiment words from domain-specific corpora or for creating entirely new sentiment lexicons [30]. However, its effectiveness depends on the availability of a sufficiently large and representative corpus [30].

Semantic Orientation and Pointwise Mutual Information

An unsupervised algorithm was proposed by Turney [1, 4], who introduced a method based on Semantic Orientation (SO). The algorithm proceeds in three steps: first, extracting two consecutive words whose Part-Of-Speech (POS) tags follow a specific patterns; second, estimating the polarity of adjectives and adverbs using Pointwise Mutual Information (PMI); and third, aggregating individual polarities to classify the entire review [1].

PMI measures the statistical dependence between a phrase and reference words (typically *"excellent"* and *"poor"*) based on their co-occurrence in a corpus or across the web [1]. A sentence receives positive SO when it has stronger associations with *"excellent"* and negative SO when associated with *"poor"* [4]. Rothfels and Tibshirani extended this approach by using two seed reference sets rather than single words, improving domain adaptability [4].

Sentiment Lexicons

Tools such as WordNet-Affect and SentiWordNet provide foundational lexical resources for sentiment analysis [30].

- **WordNet** organises nouns, verbs, adjectives, and adverbs into synsets (sets of cognitive synonyms), each representing a distinct concept [30]. WordNet 3.0 contains over 117,000 synsets, with semantic relations connecting them [30]. WordNet is an example of a well-known semantic network where definitional networks focus on "is a" relationships between a concept and a newly defined sub-type, supporting the rule of inheritance for copying properties from a super-type to all its sub-types [7].

- **SentiWordNet** builds upon WordNet by assigning three scores to each synset: positivity (P), negativity (N), and objectivity (O), which sum to 1 [30]. Version 3.0 covers 147,306 synsets and has been widely adopted due to its extensive coverage of opinion words [30]. However, SentiWordNet is sensitive to noise in datasets, particularly from social media, and does not account for internet slang or emoticons [30]. Lexicons generated with automatic methods often include neutral words, introducing noise in the detection of subjectivity [7].

- **Valence Aware Dictionary and sEntiment Reasoner (VADER)** [18] is an effective rule-based tool, which combines a sentiment lexicon with grammatical and syntactical cues such as intensifiers, negations, and punctuation [30]. VADER was specifically developed for social media contexts and incorporates acronyms (e.g., *"LOL"*), slang, and emojis [30]. The dictionary comprises approximately 7,500 features, drawing from existing lexicons including Linguistic Inquiry and Word Count (LIWC), Affective Norms for English Words (ANEW), and the General Inquirer (GI) [30]. VADER offers several advantages: it works effectively with social media text, requires no training data, supports emoji sentiment classification, and achieves high processing speed [30].

- **TextBlob** provides another accessible lexicon-based tool, offering simple polarity scoring with preprocessing capabilities through the Natural Language Toolkit (NLTK) [30].

Empirical Comparisons

Comparative studies have evaluated these lexicons across various domains. Nursal et al. [30] compared WordNet, SentiWordNet, TextBlob, and VADER on Malaysian high-rise property reviews, using human-annotated data as a baseline. Their findings revealed that all four lexicons identified a higher proportion of positive reviews than negative or neutral ones. WordNet recorded the highest number of positive sentiments, followed

closely by SentiWordNet and VADER [30]. For negative sentiment detection, VADER proved most effective, while for neutral sentiment, VADER again detected the highest number of reviews [30].

In terms of classification accuracy against human annotation, VADER achieved the best performance with 78.2% accuracy, 74.3% precision, 90.9% recall, and an F_1-score of 81.7% [30]. TextBlob ranked second, followed by SentiWordNet and WordNet with comparable performance around 69-70% accuracy [30].

However, misclassifications occurred due to several factors: limited vocabulary coverage, difficulty detecting sarcasm, word ambiguity, and challenges with misspelled or abbreviated terms [30].

Handling Sentiment Shifters and Composition

Effective lexicon-based systems must account for sentiment shifters-linguistic elements that alter the polarity of words [1, 4]. Negations (e.g., *"not good"*) reverse semantic polarity, while intensifiers (e.g., *"very good"*) modify the degree of sentiment [1]. Sophisticated approaches incorporate three pillars: a sentiment lexicon containing words, phrases, idioms, and composition rules; a set of rules for handling shifters and contrastive clauses (e.g., *"but"*); and sentiment aggregation functions derived from parse trees [1].

Preprocessing Considerations

Preprocessing requirements vary across lexicons. While VADER can handle capitalization, punctuation, and colloquial expressions with minimal preprocessing, tools such as TextBlob, SentiWordNet, and WordNet require rigorous cleaning [30]. Typical preprocessing steps include tokenization, lowercase conversion, spell-checking, stemming, stopword removal, part-of-speech tagging, lemmatization, and noise removal [30]. Normalization of abbreviations and acronyms is particularly important for social media text, as inconsistent vocabulary can degrade accuracy [30].

Advantages and Limitations

The main advantage of lexicon-based methods is that they require minimal effort for manual data annotation and do not need training datasets, making them readily applicable across domains [30]. They are particularly competitive when labelled data is scarce [1]. Domain dependency remains a challenge, as sentiment lexicons developed for one domain may perform poorly in another [4]. Additionally, lexicon-based methods struggle with sarcasm, ambiguity, and the evolving nature of language, particularly on social media platforms [30].

2.2.2 Classic Machine Learning Models

With the availability of labelled datasets, supervised machine learning became the dominant paradigm for both coarse-grained and fine-grained sentiment analysis [1]. Unlike lexicon-based methods, supervised approaches require annotated training data but can learn complex patterns from feature representations.

Feature Engineering

Generally text is first converted into a numerical representation, typically a Bag-of-Words (BoW) model where each document is represented by a vector of word frequencies. Variants include n-grams (contiguous sequences of words) and Term Frequency-Inverse Document Frequency (TF-IDF) weighting, which down-weights common words [4].

Beyond these basic features, supervised approaches commonly employ syntactic features such as POS tags and dependency relations, as well as semantic features including sentiment words, phrases, and sentiment shifters [1]. Feature engineering is the key to building effective sentiment analysis classifiers [1].

Sentence-level analysis is important because it permits a fine-grained view of different opinions expressed [7]. Sentence-level features can be in the form of n-grams, POS tags, or location based features [7]. Studies examining the different types of sentence features and their comparative effectiveness in SA concluded that features such as length or rhetorical segments in text have little effect in accuracy and could sometimes decrease sentiment accuracy; similarly, *bi*-grams perform well without the need for POS features [7].

Classification Algorithms

Classifiers such as Naïve Bayes (NB), Logistic Regression (LR), and Support Vector Machines (SVM) are then trained on these feature vectors. SVM and NB, in particular, have been shown to be the most effective classifiers for sentiment analysis tasks [4]. Extracted features are used to train state-of-the-art such as NBSVM (Naive Bayes SVM), which relies on the NB assumption that features are conditionally independent given the class label [7].

- **Support Vector Machines (SVMs)** perform well on high-dimensional sparse text data due to their margin-maximization objective and have been widely adopted for sentiment classification [4].

- **Naïve Bayes (NB)** offers simplicity and efficiency, often achieving competitive results despite its strong independence assumptions [4].

Empirical Findings

- Pang et al. [32] was the first to apply machine learning to movie review classification, showing that binary feature representation outperforms frequency-based representations.
- Pak and Paroubek [31] proposed sub-graph representations from syntactic dependency trees, showing that such structured features avoid information loss associated with pure n-gram models and improve SVM performance on French reviews.
- For Arabic sentiment analysis, Rushdi-Saleh et al. [38] introduced the Opinion Corpus for Arabic (OCA) of movie reviews and evaluated NB and SVM classifiers, achieving promising results in comparison with Pang et al. [32]. It is noteworthy that the use of n-gram models overcomes the use of the unigram model [38].
- In the Chinese domain, NB was found to outperform SVM for mobile review classification [52].
- Feature reduction techniques such as Principal Component Analysis (PCA) have been shown to improve both SVM and NB performance [46].
- Hybrid approaches combining NB with Genetic Algorithms (GAs) and rating-based features augmented with n-grams have also demonstrated performance gains [15].
- For dialectal Arabic tweets, replacing dialectal words with Modern Standard Arabic equivalents improved classifier accuracy by up to 3% [13].

The sources collectively indicate that appropriate data preprocessing and feature modeling directly impact classifier performance in sentiment analysis.

Coarse-Grained versus Fine-Grained Supervised Learning

A distinction exists between supervised learning applied at coarse granularities (document and sentence level) versus fine-grained aspect-based analysis [1].

- Features suitable for document and sentence level analysis are typically target-independent and thus inapplicable to aspect-level tasks, where the core objective is identifying opinion targets [1].
- Aspect-level supervised learning therefore requires either target-dependent feature generation or mechanisms to determine the application scope of sentiments that cover specific entities or aspects within a sentence [1].
- Additionally, aspect-level supervised learning demands annotated datasets containing labeled aspects and non-aspects, which are more costly to produce than document-level polarity labels [1].

2.2.3 Deep Learning for Sentiment Analysis

Deep learning has become very influential for SA by offering several advantages over traditional machine learning approaches [39, 43]. Unlike methods that rely on manually crafted features, deep learning models learn implicitly from the training data, eliminating time-consuming and potentially error-prone feature engineering [43]. They have the ability to adapt to new data, which leads to more robust and generalizable models [39]. This feature learning capability is particularly beneficial for large-scale sentiment analysis tasks involving social media posts or customer reviews, where deep learning algorithms excel at processing vast amounts of data due to their parallel processing capabilities [39].

Studies have consistently shown that deep learning models achieve superior accuracy compared to traditional methods such as SVM and NB [4, 28, 39, 43]. Furthermore, deep learning offers versatility across various sentiment analysis tasks, from basic polarity classification to aspect-based analysis and emotion detection [39].

These advantages come with substantial computational costs [41]. Deep learning models typically require specialized hardware such as Graphics Processing Units (GPUs) or Tensor Processing Units (TPUs) for training, leading to significant financial expenses for hardware, electricity, or cloud compute time, as well as a considerable environmental impact due to carbon emissions [41]. Modern deep learning architectures require GPU acceleration and an order of magnitude more training time than traditional approaches—which run efficiently on a standard CPU and perform inference nearly two orders of magnitude faster [28]. Deep learning models are data-hungry and perform poorly when labeled data is scarce [23]. Another issue is the difficulty of interpreting the predictions, as the models are often black boxes [23].

Word Embeddings

A fundamental shift in neural Natural Language Processing (NLP) came with the introduction of word embeddings [11]. Unlike sparse BoW representations, embeddings map words to dense continuous vectors that capture semantic similarity [11]. Two widely used embedding models are Word2Vec and GloVe [11, 34]. These pre-trained representations allow models to use semantic knowledge learned from massive corpora, providing a richer input representation than traditional feature engineering [4, 11, 39].

Convolutional Neural Networks

Convolutional Neural Networks (CNNs), originally developed for computer vision, were successfully adapted to NLP tasks by Collobert and Weston in 2008 [10]. The CNN architecture for text consists of three main layers: an input layer that receives tokenized text, a feature extraction layer comprising convolutional and pooling operations, and a classification layer with fully connected networks [10].

For sentiment analysis, CNNs excel at learning contextual features through filters that capture local patterns such as key phrases and n-gram indicators of sentiment [39]. One advantage of CNNs is their efficiency: compared to fully connected networks, they have fewer parameters, leading to faster training times [39]. Paredes-Valverde et al. [33] proposed a deep learning approach combining pre-processing, word embeddings, and a CNN model, demonstrating that CNN outperformed traditional SVM and NB classifiers. CNNs have limitations when dealing with long-term dependencies in text data, often requiring deep architectures that become computationally expensive [39].

Recurrent Neural Networks

Recurrent Neural Networks (RNNs) depart from feed-forward architectures by incorporating a memory element that retains information from previous computations [39]. This makes them naturally suited for sequential data such as text, as they can model sequences of varying lengths and capture long-range dependencies [39]. Standard RNNs suffer from vanishing and exploding gradient problems when processing long sequences, which hinders their ability to learn effectively from extended dependencies [39]. Despite these limitations, RNNs have demonstrated exceptional performance in SA tasks, with experimental results continuing to support their effectiveness [39].

Long Short-Term Memory and Gated Recurrent Unit

To address the vanishing gradient problem, Hochreiter and Schmidhuber (1997) [16] proposed the Long Short-Term Memory (LSTM) network. LSTMs incorporate a gating mechanism that allows memory cells to store information for longer durations and retrieve past computational results as needed [16]. The architecture employs three gates: an input gate controlling what information is stored, a forget gate determining what to discard, and an output gate regulating information passed to the next step [16]. Bidirectional LSTMs (BiLSTMs) process information in both directions, which is particularly valuable for identifying aspects and entities contributing to overall sentiment [39].

The Gated Recurrent Unit (GRU) offers a simpler alternative with only two gates: a reset gate that combines new input with previous information, and an update gate that controls retention and discarding [9]. GRUs provide computational efficiency comparable to LSTMs while maintaining similar effectiveness; with bidirectional variants (Bi-GRUs) also available [39].

Tang et al. [42] introduced neural network models combining convolutional and recurrent architectures for DLSA. The framework first learns sentence representations using either CNN or LSTM, then encodes the semantics and relations using gated RNNs [42]. Experiments on IMDb and Yelp datasets show that LSTM outperforms multi-filtered CNN for sentence representation [42].

Transformer Architectures

A significant breakthrough in deep learning for NLP came with the transformer architecture and its pretrained variants [45]. Bidirectional Encoder Representations from Transformers (BERT), developed by Google in 2018 [12], revolutionized language understanding through its bidirectional architecture that considers both preceding and following words in context [12]. BERT leverages a self-attention mechanism to capture relationships between words within a sentence [12]. It follows a two-step approach: pretraining on massive text corpora using masked language modelling and next-sentence prediction, followed by fine-tuning on downstream tasks such as SA [12]. BERT can tokenize text into subword units, handling various linguistic complexities, and has set new benchmarks across numerous NLP domains [12].

For sentiment analysis, fine-tuned BERT models achieve state-of-the-art results [21]. DistilBERT, a distilled version of BERT that is smaller, faster, and retains most of BERT's language understanding capabilities, has been shown to perform well on SA tasks [21].

Emerging Architectures

- **Large Language Models (LLMs)** are trained on massive and diverse text corpora, enabling them to understand complex language patterns, context, and even cultural references crucial for accurate sentiment analysis [39]. LLMs excel at tasks like sarcasm detection, where literal meaning differs from underlying sentiment, and can be fine-tuned for specific domains with minimal resources [39].

- **Graph Neural Networks (GNNs)** represent another emerging direction, designed to work with data represented as graphs where nodes represent entities (such as words) and edges represent relationships [39]. GNNs capture complex interdependencies between words, making them particularly valuable for ABSA where sentiment toward different aspects must be identified [39]. They can also integrate external knowledge bases such as sentiment lexicons and word ontologies, achieving more nuanced understanding [39].

Architecture Choice

The choice of the optimal architecture depends on the specific application and dataset. Recent research indicates that RNN variants (LSTM, GRU) outperform CNNs on many sentiment analysis tasks—largely because they excel at capturing long-range dependencies, whereas CNNs offer efficiency for local pattern detection [39]. Emerging architectures, such as LLMs and GNNs, have shown promising results for complex sentiment analysis problems [39].

2.3 Granular Computing

Granular computing is a paradigm that processes complex information by treating data as aggregations known as information granules, the process of constructing them is known as Information Granulation (IG) [3]. The granules emerge from abstracting raw data into knowledge and appear in diverse forms: time intervals (hours, days, weeks), image regions where clusters of pixels form recognizable objects, and words in natural language [3]. The human cognitive capacity is limited, people naturally partition complex information into simpler blocks based on shared properties, GrC therefore exploits the human ability for problem-solving by operating at multiple levels of abstraction [36].

2.3.1 Formalization and Core Principles

An information granule is formally defined as a collection of elements grouped by indistinguishability, similarity, or functional proximity [20]. Granules can be arranged hierarchically: coarse granules provide a high-level overview, while finer granules reveal detailed information [20].

Three fundamental concepts underpin GrC, as originally articulated by Zadeh [51]:

Granulation: decomposition of the whole into parts,

Organization: integration of parts into the whole,

Causation: association of causes and effects.

Concrete mechanisms for granulation are often seen in computing and human reasoning [3]. In computer systems, memory resources are granulated by adopting memory pages as basic operational chunks; swapping techniques then enable efficient access to individual data items [3]. When describing a problem, people tend to avoid numbers, instead using aggregates and constructing if-then rules that operate on those aggregates [3]. Although the physical world is inherently analog, computers perform digital processing [3]. Furthermore, all data compression mechanisms represent instances of information granulation [3].

2.3.2 Three-Way Decisions

Three-way decision (3WD) is a theory of thinking, problem solving, and information processing that relies on the use of three parts, three elements, or three levels [50]. It extends the classical binary (two-way) approach (typically acceptance or rejection) by introducing a third option: non-commitment, deferment, or uncertainty [14, 50]. In many real-world situations, making an immediate decision is unwise when evidence is insufficient; 3WD provides a principled way to withhold judgment until more information can be obtained [50].

The idea of trisecting a whole into three parts and then acting upon each part is a common human practice, evident in temporal (past-present-future), spatial (left-middle-right), volumetric (small-medium-large), and judgmental (acceptance-noncommitment-rejection) divisions [14]. Such a trisection can be formally represented by three pairwise disjoint subsets whose union is the universal set [14]. Its cognitive basis lies in the limited capacity of human short-term memory, which makes the number three a natural anchor-simple enough to process, yet richer than binary thinking [50]. Consequently, thinking in threes appears across many disciplines, from linguistics (three grammatical tenses) to computer science (three-level database architectures) [50].

A general model of 3WD is the Trisecting-Acting-Outcome (TAO) framework [50]:

Trisecting: dividing a whole into three pairwise disjoint parts,

Acting: devising strategies to process each of the three parts,

Outcome: evaluating the effectiveness.

The connection between three-way decision and granular computing is deep and natural [14, 50]. Both paradigms are human-inspired and rely on abstraction: granular computing builds information granules at multiple levels, while three-way decision uses three granules (or three levels) as a particularly effective cognitive structure [50]. In fact, sequential three-way decisions arise naturally from multiple levels of granularity [14]. At a higher (coarser) level of granularity, one may defer decisions about some objects to a lower (finer) level where additional detailed information becomes available [14]. This interplay has led to the integration of three-way decision and granular computing into what is sometimes called three-way granular computing [14, 50].

2.3.3 Prototype Theory and Conceptual Spaces

Prototype theory is another term from the field of cognitive sciences; it states that natural categories are organized around prototypical examples, where certain members of a category are judged to be more representative than others [11]. This theory emerged from experimental studies on human categorization and has become a foundational concept in cognitive psychology and linguistics [11].

Conceptual spaces provide a geometric framework in which concepts correspond to convex regions within a multidimensional space [11]. In such a space, each dimension represents a measurable quality or feature, and points within a region stand for individual instances of a concept [11]. The prototype of a category is then identified as a point, often the centroid, inside its corresponding region [11]. A key property is that natural properties form convex regions; meaning that if two points belong to the same concept, all points between them also belong to that concept [11].

This geometric view aligns naturally with granular computing [11]. A concept can be seen as an information granule whose members are semantically similar objects, and the prototype serves as the granule's central representative [11]. The Voronoi tessellation of a conceptual space, obtained by partitioning the space according to the closest prototype, provides a constructive way to generate such granules [11]. Thus, conceptual spaces offer a cognitive grounding for granular computing, bridging the subsymbolic level of distributed representations (e.g., word embeddings) and the symbolic level of discrete concepts [11].

2.3.4 Granular Computing for Text Classification

Granular computing has been applied to a wide range of text classification tasks, including topic categorization, spam detection, sentiment analysis, and hate speech identification [5, 25, 48]. The core idea is to transform raw textual data into information granules at multiple levels of abstraction, thereby reducing dimensionality, improving interpretability, or enabling advanced decision-making. The most relevant contributions are reviewed below, grouped by their focus.

Relational Methods

Qiu et al. [36] propose an early granular computing framework for text classification that constructs a granule library containing feature granules (keywords with class sets), effective decision granules (feature subsets that uniquely determine a class), and association degrees between features and classes. A new document is converted into an information granule by combining its feature granules, then matched against decision granules via weighted similarity for classification [36]. The authors note that the method has not been tested in practical applications and requires further investigation of similarity computation and inclusion degrees [36].

Data-Level Methods

Chen et al. [8] propose a 3WD model for binary SA that selects distinct feature subsets for the positive and negative classes, rather than using a uniform representation. During testing, confident samples are classified using their class-specific optimal feature subset, while uncertain samples fall into a boundary region and are processed with the full feature set [8]. Experiments on IMDb and Amazon show improved accuracy over chi-square and PCA-based feature selection, albeit with increased computational overhead [8].

Behzadidoost et al. [5] propose a Granular Computing-based Deep Learning model (GCDL) for cyberbullying and hate speech detection. The model generates two types of granules: textual granules, constructed by a "reverse pancake-scramble" algorithm that flips characters and reverses vowels, and numerical granules, extracted via ten rule-based features (e.g., hashtags, stop words, emojis) [5]. Textual granules are processed by a

stacked CNN-LSTM, while numerical granules feed into a stacked BiLSTM-SVM—the first such combination in GrC [5]. An attention mechanism merges both outputs for final classification [5]. On three datasets (cyberbullying and two hate speech collections), GCDL achieves 88.74%, 89.62%, and 70.85% accuracy, outperforming state-of-the-art models [5]. Ablation studies confirm that both granule types contribute positively, and the granulation process does not exceed the original dataset size, avoiding imbalance [5].

Three-Way Decision Methods

In dynamic environments where data streams arrive over time, static sentiment classifiers become outdated [48]. Yang et al. [48] propose a temporal-spatial three-way granular computing framework that incrementally updates the model as new data arrives [48]. At each time step, the model classifies samples into three regions: positive, negative, and boundary. Confident samples are accepted or rejected immediately, while uncertain ones are deferred to the next level, where finer granularity (more data and updated parameters) enables more reliable decisions [48]. This sequential three-way strategy reduces misclassification and time costs without sacrificing accuracy, as shown on IMDb, Yelp, and Amazon datasets using TextCNN, TextRNN, TextRCNN, and fastText [48].

For open topic classification where test data may contain unseen classes, the authors propose Three-Way Multi-Granularity learning towards Open topic classification (TWMG-Open) [49]. The model operates at three granular levels: first, known topics are separated from unknowns using adaptive spherical decision boundaries; second, unknown samples are clustered via three-way Density-Based Spatial Clustering of Applications with Noise (DBSCAN) to discover hidden classes; third, high-confidence clusters are added to a knowledge base for continual learning [49]. Experiments on Banking77, StackOverflow, and TC20 show that TWMG-Open outperforms traditional baselines, especially when the proportion of known classes is high [49].

Bag-of-Concepts Methods

Moving beyond word-level representations, several authors have explored Bag-of-Concepts (BoC) approaches that aggregate semantically related tokens into higher-level information granules [6, 11, 35]. De Santis et al. [11] directly apply prototype theory: word embeddings are clustered into Voronoi regions, and each document is represented by a histogram of cluster memberships. This concept-level representation can rival word-level methods for text processing tasks, while offering better interpretability [11].

Capillo et al. [6] present a granular computing framework that mines m-grams (short phrases) as information granules for text categorization. Each word in an m-gram is mapped to a vector using fastText embeddings, and the resulting sequence of vectors is compared using a custom edit distance that operates on word vectors rather than

characters [6]. A per-class clustering procedure extracts representative m-grams, which serve as alphabet symbols (concepts) [6]. Documents are then transformed into symbolic histograms counting the occurrences of these concepts, and the histograms are classified by an SVM [6]. A genetic algorithm optimizes the system's hyperparameters and selects the most discriminative symbols [6]. Experiments on scientific abstracts (four classes) achieve about 93% accuracy, and on an SMS spam dataset about 95% accuracy [6]. Importantly, the selected symbols are interpretable phrases (e.g. *"string field theory"* for theoretical physics, *"customer care"* for spam), demonstrating that the BoC approach yields both high performance and explainable models [6].

Possemato et al. [35] extend the BoC approach to handle unbalanced datasets by extracting n-grams and then running a per-class granular mining algorithm known as Reinforcement Learning Granular Approach for DIscrete Sequence (RL-GRADIS) to obtain representative symbols for each category [35]. The union of these per-class symbols forms the final alphabet, and documents are converted into symbolic histograms by counting occurrences of each symbol [35]. An SVM with a Gaussian kernel then classifies the histograms, and a GA performs feature selection to retain only discriminative symbols [35]. Experiments on the unbalanced Reuters-21578 dataset (ten categories) show that the per-class strategy improves results compared to the standard unsupervised version, demonstrating better handling of sparse classes [35]. The extracted symbols remain interpretable [35].

Hierarchical Methods

Liu and Cocea [25] propose a hierarchical, fuzzy rule-based system for SA to address the high dimensionality inherent in traditional BoW models. Their approach decomposes documents into sub-levels (such as sections, paragraphs, and sentences) and subsequently groups these localized fragments into similar clusters [25]. The model extracts BoW features from "small and simple" cluster vocabularies, effectively bypassing the sparsity of a global matrix [25]. The system then applies fuzzy logic to score these local granules, aggregating the membership degrees upward to classify the whole text [25]. This method builds on their prior baseline experiments; while those early models achieved competitive accuracy, this hierarchical iteration successfully mitigates their large dimensionality while preserving the transparency of natural-language "if-then" rules [25].

Huang et al. [17] propose a Multi-granular Joint Sentiment-Topic model (MgJST) that extends sentiment topic models based on Latent Dirichlet Allocation (LDA) by introducing two additional layers: a sentence layer between documents and words, and a sentiment layer between documents and topics [17]. Unlike previous models such as Joint Sentiment Topic (JST), which treat a document as a single topic mixture, MgJST assigns a distinct topic distribution to each sentence within a document, capturing finer-grained structural knowledge [17]. The model is a five-layer hierarchical architecture that uses

sampling for parameter estimation and can determine document-level sentiment polarity via a voting rule [17]. Experiments on seven review datasets show that MgJST significantly outperforms unsupervised baselines in sentiment detection and achieves competitive performance with supervised deep learning methods [17]. The extracted topics are interpretable, demonstrating the model's ability to reveal opinionated topics at multiple granularities [17].

In Granular Computing-based Multi-level Interactive Attention (GC-MIA) networks proposed by Li et al. [22] each word is treated as an indivisible granule, and a three-level interactive attention mechanism refines the representations of both the target and its context. The first level uses matrix-based interactive attention to capture pairwise word interactions; the second level incorporates Euclidean distance to encode semantic closeness; the third level applies interactive attention between child granules (individual words) and parent granules (the aggregated target or context) to assign sentiment-relevant weights [22]. The final context and target granules are merged into an ancestor granule and fed to a softmax classifier [22]. On the SemEval 2014 laptop and restaurant datasets, GC-MIA achieves accuracies of 73.7% and 79.0%, respectively, outperforming LSTM, Time Distributed LSTM (TD-LSTM), and the original Interactive Attention Networks (IANs) [22]. A case study further visualises that the model assigns higher attention weights to context words semantically or sentimentally relevant to a given target, illustrating the explanatory power of the multi-level granular design [22].

Cascade Methods

Varshney and Baral [44] describe a sequence of models of increasing capacity (e.g., BERT-mini, medium, base) to improve both computational efficiency and prediction accuracy. An input is first processed by the smallest model; if its confidence exceeds a threshold, the prediction is output immediately [44]. Otherwise, the instance is passed to the next larger model. This avoids expensive inference on easy instances [44]. Experiments on ten Natural Language Understanding (NLU) datasets show that cascading saves up to 88.93% of computation cost while matching or even exceeding the accuracy of the largest model alone (e.g., a 2.18% accuracy improvement on CommitmentBank) [44]. The method requires no architectural modifications and adds minimal storage overhead [44].

Online cascade learning proposed by Nie et al. [29], is a framework where smaller models are updated continuously over a data stream by imitating the outputs of a LLM expert. The system is formulated as an episodic Markov decision process with a deferral policy that decides, for each incoming query, which cascade level to invoke [29]. Smaller models (e.g., LR and BERT) learn online from the LLM's annotations, improving over time and gradually handling an increasing proportion of queries [29]. This significantly reduces the number of calls to the expensive LLM, cutting inference costs by up to 90% while maintaining accuracy comparable to using the LLM alone [29]. The approach re-

quires no human-labeled training data and remains robust under distribution shifts in input length or category, as demonstrated on sentiment analysis (IMDb), hate speech detection, emotion classification, and fact-checking benchmarks [29].

2.3.5 Gap Analysis and Contribution

Gap Analysis

The preceding sections have surveyed sentiment analysis at three levels (document, sentence, aspect) and highlighted persistent challenges: contextual polarity ambiguity, sarcasm and irony, negation handling, implicit opinions, and vocabulary evolution [4, 39]. Existing methods fall into three broad families: lexicon-based (interpretable but fragile), classic machine learning (efficient but feature-sensitive), and deep learning (accurate but computationally expensive). Granular computing offers a conceptual bridge between these families, but existing GrC work for text classification has not fully exploited the trade-offs specific to sentiment analysis.

Gap 1: Granular computing approaches have not been systematically evaluated for sentiment analysis. Methods such as BoC have shown promise in tasks such as topic categorization and spam detection [6, 35]. However, the BoC paradigm has never been assessed for polarity detection, and no prior work has explored whether injecting external sentiment knowledge (e.g., lexicon scores) during granulation can improve concept quality. Moreover, the feature space in text classification is highly sparse; Chen et al. [8] demonstrated that class-specific feature selection can improve SA performance, pointing to the potential benefits of granular vocabulary reduction. A systematic investigation that evaluates how granular methods can affect sentiment classification is therefore missing.

Gap 2: No efficient hybrid system design that combines lightweight classifiers with transformer models for sentiment analysis. Sequential three-way decision frameworks (e.g., Yang et al. [48]) defer uncertain samples to finer granularities, but they typically apply the same deep architecture at all levels, incurring high costs even for easy samples. In sentiment analysis, many reviews are clearly positive or negative and could be handled by a computationally efficient base model, while only ambiguous or sarcastic cases require a high-capacity specialist transformer. Existing work does not provide a method to form such cascade, nor does it determine what proportion of instances should be deferred—i.e., how to select the optimal threshold that balances coverage and accuracy.

Contribution

This work addresses the two gaps with the following contributions:

Contribution 1: Granular Computing Methods for Sentiment Analysis. An evaluation of granular computing techniques is conducted for binary sentiment analysis. Two complementary approaches are examined: (1) statistical Z-score pruning of n-gram features, which discards non-discriminative vocabulary based on class-separability, and (2) semantic clustering into concept granules using `Sentence Transformer` embeddings, optionally augmented with `VADER` sentiment scores. The experiments on the IMDb dataset demonstrate that Z-score pruning not only preserves or improves classification accuracy—the pruned, stemmed token representation reaches 91.41% accuracy, while LR similarly benefits—but also greatly reduces feature space dimensionality. For the $(1, 3)$ n-gram range, pruning with a threshold of $|Z| > 2.0$ compresses the vocabulary from 139,015 to just 36,350 features. This reduction mitigates the memory explosion that would otherwise occur with deeper ranges n-gram ranges. In contrast, concept-level representations, though producing interpretable clusters fail to surpass the unclustered n-gram baseline, even with sentiment-weighted embeddings. These findings delineate clear empirical boundaries for GrC in SA: token-level pruning is both beneficial and practically necessary for scaling to higher-order n-grams, while semantic aggregation may destroy crucial polarity cues.

Contribution 2: Cascade Granular Computing for Sentiment Analysis. A cascade ensemble architecture is proposed that follows the three-way decision paradigm at the model level rather than the feature level. A lightweight base model processes all instances; those classified with high confidence (falling outside a threshold-defined boundary region) are accepted immediately, while uncertain instances are deferred to a specialist deep learning model. The thesis provides a systematic method for selecting the optimal deferral threshold by sweeping over the validation set and maximizing hybrid accuracy, and evaluates three specialist variants (generic BERT, fine-tuned BERT, and SVM). Empirical results on the IMDb dataset demonstrate that the cascade achieves a macro F_1-score of approximately 92.3%, outperforming both standalone baselines, while delegating only 15-20% of instances to the expensive model.

Chapter 3

Granular Cascade System for Sentiment Analysis using Enumerations

3.1 Overview of the Pipeline

The system operates at three levels. First, the raw data undergoes preprocessing to ensure consistency, and to create a basis for the experiments. Next, GrC approaches are utilized to find the most informative features, and the most robust representations and models. Finally, a cascade ensemble based on the 3WD paradigm is introduced: a traditional machine learning model provides rapid predictions for clear-cut cases, while a deep learning model is reserved for ambiguous instances that the base model cannot classify with high confidence.

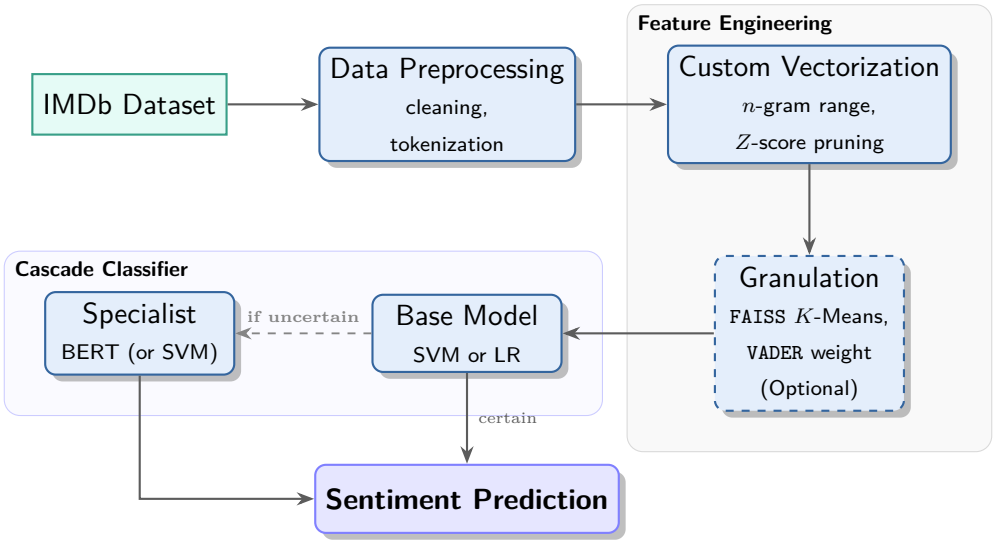


Figure 3.1: High-level pipeline of the cascade sentiment analysis system.

3.2 Data Preprocessing

While the original IMDb dataset is perfectly balanced and pre-divided into equal training and testing sets, this study merges the sets to perform data deduplication based on exact matches of the raw review text. This process identified and removed 418 duplicate entries, resulting in a refined, unique dataset of 49,582 reviews. The dataset is shuffled and re-divided into a training set of 24,791 reviews (approx. 50%), a validation set of 17,353 reviews (approx. 35%), and a testing set of 7,438 reviews (approx. 15%). Stratification is employed to maintain a balanced distribution of positive and negative sentiments across all sets. The textual data is processed in parallel using a process pool executor. Each review is processed sequentially as follows:

1. **Unicode normalization** converts the original text to Normalization Form Compatibility Composition (NFKC) form to ensure consistent character representation.
2. **Typographical unification** uses a translation table to replace uncommon characters with their ASCII equivalents (e.g., *“. . . ” → “...”* or *“— ” → “- ”*).
3. **Tag removal** replaces any substrings within the HTML tags <> with a space.
4. **Whitespace condensation** collapses multiple whitespace characters into a single space, and strips leading/trailing spaces.
5. **Contraction expansion** is implemented using `contractions.fix` function to expand English contractions (e.g., *“don’t” → “do not”*).
6. **Tokenization** produces `tokens_cased` using NLTK’s `word_tokenize`.
7. **Lowercasing** converts each token in `tokens_cased` to the lowercase form, resulting in `tokens_lower`.
8. **POS-filtered tokenization** replaces the standard stop-word removal (Sec. 3.2.1); the expanded text (`tokens_lower`) is filtered based on the POS tags; the result is stored as `tokens_filtered`.
9. **Alphabetic filtering** removes any character that is not a letter (a-z, A-Z) from `tokens_lower`; tokens that become empty after this removal are discarded. The result is `tokens_letters`.
10. **Stemming** produces `tokens_stemmed` from `tokens_lower` by applying the Porter stemmer.
11. **Lemmatization** applies the WordNet lemmatizer to `tokens_lower`, producing `tokens_lemmatized`.

After processing, each review is stored as a dictionary containing the various token representations. The dictionaries are then converted to Pandas DataFrames with a consistent column order and saved as Parquet files (`train.parquet`, `val.parquet`, `test.parquet`). This format enables selective loading of representations in subsequent experiments.

3.2.1 Stop-Word Removal

During the POS-filtering step, the dimensionality of the n-gram feature space can be reduced by removing words that carry little sentiment value—commonly termed as stop words. However, because this step is inherently subjective and risks discarding subtle contextual cues, the standard stop-word removal using the English stop-word list provided by scikit-learn's `CountVectorizer` is bypassed as that list contains words such as *"well"*, whose presence may be informative (e.g., *"well filmed"*).

Instead, stop-word filtering is kept to a minimum by utilizing a custom POS-based approach. The approach relies on the tags assigned by the spaCy library: coarse-grained Universal POS (UPOS) tags[1] and fine-grained Penn Treebank tags[2]. (To avoid computational overhead parser and Named Entity Recognizer (NER) of the SpaCy's English model are disabled.)

The POS filtering removes tokens that satisfy any of three specific criteria:

- **Auxiliary verbs (except modals)**: tokens with the coarse tag `AUX` (auxiliary) are removed, unless their fine-grained tag is `MD` (modal auxiliary). Thus, words such as *"is"*, *"are"*, *"was"*, and *"have"* are removed, while modals like *"can"*, *"will"*, and *"must"* are retained.

- **Determiners**: tokens with the coarse tag `DET` (determiner) are removed. Examples include *"a"*, *"an"*, *"the"*, *"many"*, and *"my"*.

- **Possessive endings**: tokens with the fine-grained tag `POS` (possessive ending) are removed. This typically corresponds to the possessive clitic *"'s"*.

The application of these rules is demonstrated in Table 3.1. The listed tags include basic parts of speech such as pronouns (`PRON/PRP`), verbs and auxiliaries (`VERB/VBN`, `AUX/VBP`), adverbs (`ADV/RB`), determiners (`DET/DT`, `DET/PRP$`), adjectives (`ADJ/JJ`, `ADJ/JJR`), nouns (`NOUN/NN`, `NOUN/NNS`), adpositions (`ADP/IN`), and punctuation (`PUNCT/.`). Rows designated for removal are highlighted in **bold** to help visualize how the approach works.

[1] `https://universaldependencies.org/u/pos/`
[2] `https://www.ling.upenn.edu/courses/Fall_2003/ling001/penn_treebank_pos.html`

Table 3.1: Example of POS filtering. Tokens intended for removal are depicted in bold.

Token	Lowercased	Coarse POS	Fine-grained POS
I	i	PRON	PRP
have	**have**	**AUX**	**VBP**
got	got	VERB	VBN
myself	myself	PRON	PRP
quite	quite	ADV	RB
a	**a**	**DET**	**DT**
proud	proud	ADJ	JJ
collection	collection	NOUN	NN
with	with	ADP	IN
many	**many**	**DET**	**DT**
more	more	ADJ	JJR
titles	titles	NOUN	NNS
on	on	ADP	IN
my	**my**	**DET**	**PRP$**
list	list	NOUN	NN
!	!	PUNCT	.

3.3 Feature engineering

To evaluate the impact of vocabulary representation on model performance, the tokenized text is transformed into discrete feature spaces using combinations of unigrams, bigrams, and trigrams. However, higher-order n-grams naturally introduce massive, highly sparse feature spaces. To mitigate the curse of dimensionality, an attempt is made to employ a few dimensionality reduction techniques: custom vectorization, statistical pruning and semantic clustering.

3.3.1 Custom Vectorization

By default, scikit-learn's `CountVectorizer` removes terms listed in the `stop_words` parameter, generates n-grams, and then prunes the vocabulary according to the `min_df` and `max_df` thresholds. To faithfully evaluate the various token representations produced during preprocessing, it is essential to preserve their exact structures. Therefore, the default tokenization behavior is overridden.

At this stage, each document is a list of tokens. The lists are joined into single, space-separated strings, and a custom `space_tokenizer` is supplied to the vectorizer. This tokenizer bypasses the default regular-expression-based tokenizer and splits text strictly on spaces. The vectorizer is additionally configured with `lowercase=False` (lowercasing was already performed during preprocessing) and the desired `ngram_range` (e.g., $(1, 3)$). This customized pipeline ensures that scikit-learn's frequency-based pruning is applied directly to the finalized n-grams, effectively reducing dimensionality and promoting model

generalization. For this study, the minimum document frequency is set to 10 (`min_df=10`) to discard extremely rare n-grams, and a maximum document frequency threshold of 70% (`max_df=0.7`) is established to filter out overly omnipresent combinations.

3.3.2 Statistical Pruning

A custom statistical pruning method based on Z-scores is employed to discard neutral noise and retain only highly discriminative vocabulary. This process evaluates each feature by comparing its relative frequency between the positive class (ρ) and the negative class (η) through the following steps:

Smoothed Probabilities

First, the smoothed token-level probability of a feature in a given class κ is calculated, denoted as $s(\kappa)$. To prevent undefined logarithms resulting from zero-frequency occurrences, Laplace smoothing ($\beta = 0.01$) is applied:

$$s(\kappa) = \frac{\|\kappa\| + \beta}{T_\kappa + \beta V} \tag{3.1}$$

where:

- $\|\kappa\|$ is the number of occurrences of the feature in class κ,
- T_κ is the total number of tokens in all documents of class κ,
- V is the total size of the vocabulary.

This formulation treats each feature as a token generator—representing the probability of drawing that specific term when sampling tokens from class κ.

Computing the Z-Score

Next, the log-odds ratio of the feature's smoothed probability in the positive class versus the negative class is calculated. The Z-score normalizes this log-odds ratio by its standard error:

$$Z = \frac{\ln \frac{s(\rho)}{s(\eta)}}{\sqrt{\frac{1}{\|\rho\|+\beta} + \frac{1}{\|\eta\|+\beta}}}. \tag{3.2}$$

Thresholding

Finally, features whose absolute Z-score falls below a defined threshold (e.g., $|Z| \leq 2.0$) are discarded. This preserves only the vocabulary that demonstrates statistically significant separation in token distribution between the two classes.

3.3.3 Semantic Clustering

To address the sparsity inherent in BoW and TF-IDF representations, an optional clustering pipeline is implemented. Traditional static word embeddings such as Word2Vec are limited to individual words and cannot directly represent arbitrary multi-word n-grams. In contrast, transformer-based models like the one employed here—`all-MiniLM-L6-v2`—produce dense contextual embeddings of dimension 384 for any input text, including bigrams and trigrams. Although generating these embeddings is computationally expensive, they provide the most semantically rich representation currently available and were deliberately chosen to test whether powerful, context-aware grouping of n-grams can outperform simpler vocabulary reduction methods.

Each n-gram in the vocabulary is encoded with this Sentence Transformer model. The resulting embedding is then augmented with a sentiment score obtained from the VADER lexicon: the compound polarity score of the n-gram is multiplied by a tunable hyperparameter w and concatenated to the embedding vector. After augmentation, L_2 normalization is reapplied to restore unit length, ensuring the final vector space balances semantic proximity with sentiment polarity.

FAISS-accelerated K-Means clustering is utilized on these normalized, sentiment-aware vectors to partition the vocabulary into k discrete groups. Each cluster forms a "concept" that aggregates semantically and sentimentally related n-grams. The resulting mapping from each original n-gram to its concept identifier is stored. Concept-level feature matrices are then constructed by summing the term-frequency counts of all n-grams belonging to the same concept for each document. This produces a dense $D \times k$ matrix that compresses the original high-dimensional vocabulary while preserving discriminative information. Statistical pruning via Z-scores can be applied to these concept features in exactly the same manner as to the original units, further reducing dimensionality and focusing the representation on the most sentiment-bearing concepts.

3.3.4 Models and Training

The system employs both traditional classifiers and a distilled transformer model. All models are trained on the training set and tuned on the validation set; the chosen hyperparameters are those that maximise validation accuracy.

Traditional Baselines

An SVM and LR are employed as the computationally efficient baselines. Both are trained on the TF-IDF and BoW n-gram vectors produced by the feature engineering pipeline. Both models are fitted on the same training split and evaluated on the same validation and test sets. The training utilizes the `scikit-learn` implementations with the following settings:

Support Vector Machine. This model minimizes the squared hinge loss with L_2 regularization. The default penalty $C = 1.0$ is retained, and the solver's iteration limit is set to 10,000 to ensure convergence on high-dimensional data. Because the final cascade architecture requires calibrated probabilities, a Platt scaling calibrator (`CalibratedClassifierCV`) is wrapped around the SVM. This post-processing step maps the decision values to $[0, 1]$, producing approximate posterior probabilities for the positive class. Two kernel types are employed: a linear kernel and the Radial Basis Function (RBF) kernel, also known as the Gaussian kernel.

Logistic Regression Model. directly models $P(y = 1 \mid X)$ by minimizing the log-loss (cross-entropy) with an L_2 penalty. It is configured with the `L-BFGS` solver, $C = 1.0$, and a maximum of 1,000 iterations. Unlike the SVM, LR natively outputs probability estimates without an external calibration step.

Deep Learning Model

The specific checkpoint `distilbert-base-uncased-finetuned-sst-2-english` of DistilBERT is obtained from the Hugging Face hub; it has been pre-trained on large English corpora and already fine-tuned on the Stanford Sentiment Treebank (SST-2).

In this study, the model is further fine-tuned on the full IMDb training set. Input texts were cleaned (`text_clean`) and subsequently tokenized with the DistilBERT tokenizer, and finally truncated or padded to a maximum length of 256 tokens. Training is performed for 3 epochs with a batch size of 16, using the Adam optimizer with a learning rate of 2×10^{-5}. Mixed-precision (`float16`) computation is utilized to reduce memory consumption and accelerate training. Early stopping monitors the validation loss (patience 2 epochs), and a learning rate reduction on plateau (factor 0.5, patience 1) is applied. A stratified 20% hold-out from the training set serves as the validation partition. After training, the model outputs class probabilities through a softmax over the two logits.

3.3.5 Configuration Enumeration

The feature-engineering options define a large design space—token representations, n-gram ranges, pruning thresholds, clustering granularities, and sentiment weights. To populate this space with trained classifiers, every combination of the following axes is enumerated and evaluated on the validation set:

- **Token representation:** Cased, lowered, letter-only, filtered, stemmed, and lemmatized.
- ***n*-gram range:** $(1, 1)$, $(1, 2)$, $(1, 3)$. The default $(1, 3)$ is used for the full combinatorial sweep; narrower ranges are tested in the vocabulary-depth test.

- **Z-score threshold:** $0, 1, 2, 3, 4$, where 0 disables pruning.
- **Concept granularity (k):** 0 (no clustering), 500, 5 000, 10 000.
- **Sentiment weight (w):** 0, 1, 10, applied to the VADER score when augmenting embeddings.
- **Vectorisation type:** BoW and TF-IDF (custom implementations).
- **Classifier:** linear SVM (with Platt scaling) and LR. Note that RBF SVM is tested in Section 4.4.5, but it proved to be much slower with no accuracy benefit.

Not all combinations are independent: for example; when $k = 0$ the sentiment weight is irrelevant. For every configuration, the model is trained on the training set and its raw prediction vectors and classification reports are saved to disk. At this stage no representation is selected; the pool of candidate classifiers is merely created for subsequent cascade-oriented analysis.

3.4 Cascade Ensemble

To balance the computational efficiency of traditional machine learning algorithms with the deep understanding capacity of transformer-based models, a cascade ensemble architecture is proposed. The design follows the 3WD paradigm, where each instance is assigned to one of three regions based on the predictive confidence of a lightweight base model. Instances classified with high confidence are accepted immediately, while those falling into a boundary region of uncertainty are deferred to a specialist.

3.4.1 Architecture

The cascade operates in three stages:

1. **Initial inference:** A computationally inexpensive base model evaluates an input text X and outputs a probability score

$$p_B = P_B(y = 1 \mid X) \in [0, 1]. \tag{3.3}$$

2. **Uncertainty delegation:** A threshold $\tau \in [0, 0.5]$ is defined. The intervals $[0, \tau)$ and $(1 - \tau, 1]$ correspond to high-confidence negative and positive predictions, respectively, which are accepted without further processing. Otherwise X is placed in the boundary region if the base model's probability lies within the closed interval:

$$\tau \leq p_B \leq 1 - \tau, \tag{3.4}$$

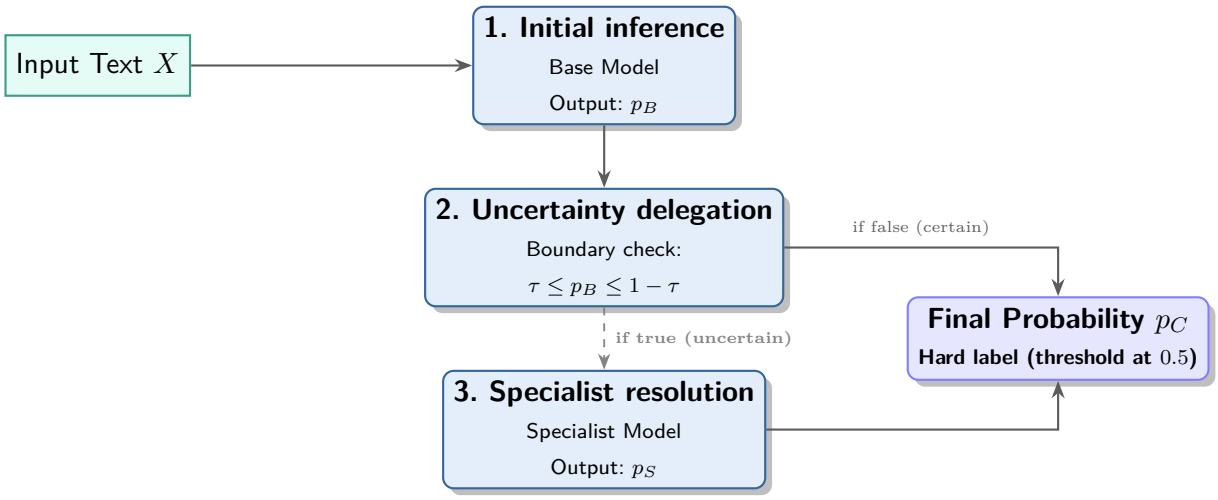


Figure 3.2: Architecture of the three-stage cascade showing initial inference, uncertainty delegation, and specialist resolution.

3. **Specialist resolution:** Instances in the boundary region are forwarded to a specialist model. Three specialist configurations are evaluated:

   - a generic DistilBERT classifier used as a black-box specialist;
   - a DistilBERT specialist fine-tuned exclusively on the boundary samples mined from the validation set;
   - an SVM specialist trained on the same boundary samples to verify whether a two-stage traditional approach would achieve satisfying results.

Let $p_S = P_S(y = 1 \mid X)$ denote the probability output of the chosen specialist. The final cascaded probability, p_C, is obtained by combining the two models:

$$p_C = P_C(y = 1 \mid X) = \begin{cases} p_S, & p_B \in [\tau, 1 - \tau], \\ p_B, & p_B \notin [\tau, 1 - \tau]. \end{cases} \tag{3.5}$$

The hard class label is derived by thresholding p_C at 0.5: values strictly greater than 0.5 are taken as positive, otherwise negative.

3.4.2 Threshold Selection

The delegation threshold τ governs the trade-off between the fraction of instances processed by the base model B (coverage) and the overall performance of the hybrid system (global accuracy). To determine both the optimal base model B_o and the optimal threshold τ_o, a joint sweep is performed on the validation set. For every candidate base model μ obtained from the configuration enumeration and for every candidate threshold $t \in [0, 0.5]$, the cascade is simulated using the generic DistilBERT. The hybrid accuracy

$\aleph(\mu, t)$ is computed as defined below, and the pair (μ, t) that maximizes this metric is selected. Let the validation set contain N instances, indexed by $\iota = 1, \ldots, N$. For a fixed candidate base model μ, denote the predicted probability for instance ι by $p_{B_\mu}^{(\iota)}$. For a candidate threshold t, the acceptance mask for instance ι under model μ is:

$$A_{\mu,\iota}(t) = \mathbf{1}\Big[p_{B_\mu}^{(\iota)} \leq t \ \vee \ p_{B_\mu}^{(\iota)} \geq 1 - t\Big]. \tag{3.6}$$

The coverage of model μ at threshold t is the fraction of instances it accepts:

$$\epsilon_\mu(t) = \frac{1}{N} \sum_{\iota=1}^{N} A_{\mu,\iota}(t). \tag{3.7}$$

The hybrid accuracy of the cascade when using model μ and threshold t is:

$$\aleph(\mu, t) = \frac{1}{N} \sum_{\iota=1}^{N} \mathbf{1}\Big[\hat{y}_{\mu,\iota}^{C}(t) = y_\iota\Big]. \tag{3.8}$$

where $\hat{y}_\iota^{C}(t)$ is the cascade's final prediction for instance ι under the given model and threshold. The optimal base model and the optimal threshold are chosen simultaneously:

$$(B_o, \ \tau_o) = \arg \max_{\mu, \ t \in [0, 0.5]} \aleph(\mu, t). \tag{3.9}$$

3.4.3 Specialist Training

Two types of specialists are trained exclusively on the validation documents whose base model prediction probability $p_B^{(\iota)}$ falls within $[\tau, 1 - \tau]$.

Specialist SVM. The same SVM architecture and hyperparameters as the base model is employed, but the model is fitted solely on the TF-IDF representations of the boundary samples. This yields a lightweight specialist that shares the feature space and decision function type of the base model while concentrating on the ambiguous region.

Specialist BERT. The basic DistilBERT model is further fine-tuned on the boundary subset. The backbone is partially frozen: the first 5 transformer layers are kept frozen, while the remaining layers and the classification head remain trainable. The specialist is trained for 10 epochs with a lower learning rate of 2×10^{-6}, batch size 16, and early stopping (patience 3 epochs). All other training settings (mixed precision, tokenizer, maximum sequence length) are identical to the basic fine-tuning. This procedure encourages the model to specialize in the ambiguous region of the decision boundary without overfitting to the relatively small hard-sample set.

3.5 Implementation

All experiments, data transformations, and statistical analyses are implemented in Python. The architecture leverages open-source libraries across the different domains of the experimental pipeline:

- **Data Management and Processing:** Core data manipulation, matrix operations, and dataset parsing are handled using `pandas` and `NumPy`.
- **Natural Language Processing:** the `contractions` library is utilized alongside `NLTK`, which provides the implementations for Porter Stemming and WordNet Lemmatization. Context-aware POS filtering is executed with `spaCy` (`en_core_web_sm`). Lexicon-based sentiment scoring is provided by `vaderSentiment`.
- **Classical Machine Learning:** the construction of BoW and TF-IDF matrices, as well as the implementation of the traditional classifiers (linear SVM and LR), are conducted using `scikit-learn`. For high-dimensional semantic clustering, `FAISS` is employed to perform GPU-accelerated K-Means clustering.
- **Deep Learning:** the deep learning components of the cascade architecture are built upon the `TensorFlow` and `Keras` frameworks. The DistilBERT model and its tokenizer are sourced from the Hugging Face `transformers` library. The generation of dense semantic vectors prior to clustering is powered by the `sentence-transformers` library.
- **Statistical Testing and Visualization:** `statsmodels` is employed to perform McNemar's tests. All graphical representations are generated with `matplotlib`, `seaborn`, and the `wordcloud` library.

The code can be found in the repository[3]. The main experiment is located in the `Experiment.ipynb` file, while the reusable structural components are in the `src/` directory, which contains the implementations of machine learning models, feature extraction methods, and utilities. In the `scripts/` directory, all steps that appear in the notebook are available as standalone scripts; these files are intended mostly for one-time use.

[3] `https://github.com/JanSupierz/Sentiment`

Chapter 4

Experiments

4.1 Datasets

For the empirical evaluation in this study, the Large Movie Review Dataset is used—commonly referred to as the IMDb sentiment dataset, introduced by Maas et al. [26][1]. This dataset serves as a standard benchmark for binary sentiment classification in natural language processing. It contains a total of 50,000 highly polar movie reviews. The acquisition of the IMDb dataset is facilitated by TensorFlow Datasets (TFDS).

4.1.1 Examples

Below are example reviews from the dataset. These examples demonstrate the text's typical length, as well as its informal and rather descriptive style.

Label: Positive

> Wasn't sure what to expect from this film. I love watching Brosnan in any movie, he's always good, but this was totally different. He's a mess, and plays being a mess very well. In fact he reminded me of myself a few times back in uni :) So yes, there's not a lot of violence, there's not a lot of action, but the dialogue is cracking, the acting is superb and very refreshing, and it's pretty funny too :) I don't think you'll come out of the cinema going "Wow! I was blown away", but you'll come out smiling having enjoyed a good film. Can't ask for more than that.

Oh, and if you're lucky, some moron will put the adverts reel in wrong and you'll get inverted, upside down, reversed (with reversed audio) adverts!! Brilliant. Nothing like watching a Jack Daniels ad where the drink goes back into the bottle from the glass with a gruff American "Twin Peaks" commentary :)

[1] `https://ai.stanford.edu/~amaas/data/sentiment/`

At first sight, the positive text tends to use words with positive connotations—such as *"love"*, *"good"*, and *"lucky"*—as well as emoticons like *":)"*. However, clearly negative words (*"mess"* or *"moron"*) can also be found, demonstrating that context is crucial.

Label: Negative

> The first half of the movie is not that bad actually. Although there's not really too much depth in the characters, the story is somehow funny and generally OK with potential to get better, which it doesn't.

In the second half things start to turn for the worse, not only for the characters in the movie, but also for the viewer, who will be basically waiting for the story to come to its obvious end.

The previous user comment mentioned: "It's a love story, a road movie, a thriller, a comedy of errors, an 80's movie and most of all, it's a Jonathan Demme movie."

Well, please allow me to rephrase that: "It's a boy gets girl story, they happen to be driving around in cars, the thrill is gone as it is all too clear how things will evolve, there wasn't any scene in this film to which one might laugh out loud, it does have some slight 80's feeling and yes, Jonathan Demme really was the director."

The negative review shows how sentiment can shift within a single text. The author begins with positive evaluations, using phrases conceding that the film is *"not that bad."*. The tone progressively deteriorates, relying on phrases that express disappointment such as *"turn for the worse"* and *"the thrill is gone."*. Furthermore, sarcasm and cynicism play a crucial role in cementing the overall negative polarity.

Possibly Mislabeled

> I first watched Kindred in 1987 along with another movie called devouring waves. I remember back then i hated them both and i have never really bothered to watch them again.

However i have recently started a crusade to collect as many 80's horror titles in their original boxed form, That have been deleted for some time. I have got myself quite a proud collection with many more titles on my list!

The Kindred although i have not as yet got a copy is high priority as all the old movies i didn't like back then, I now own and have now re-watched and think they are brilliant and the bits i do remember of the Kindred are now driving me to want to get hold of a copy A.S.A.P.

Hurray for the 80's and long live horror!

The text is labeled as negative within the dataset, but is overly positive. This discrepancy suggests that either the entire message is meant to be read as sarcastic, or the user made an error when selecting their rating.

4.2 Methodology

The aim is to define the impact of each individual step in the pipeline described in Chapter 3 and to determine if a cascade system further improves upon those results. To do so, two experiments, **Experimental Evaluation Sequence** (Sec. 4.2.1) and **Cascade System Experiment** (Sec. 4.2.2), are constructed. Both experiments consist of sub-stages where the configuration that yields the highest validation metric is considered the best setting and is adopted as the baseline for the next stage. The test set is never used until the final cascade evaluation. The experiments are outlined in the next sections.

4.2.1 Experimental Evaluation Sequence

The enumeration space defined in Section 3.3.5 is explored through a series of sub-experiments on the validation set. Each step refines the process and provides insights into its individual impact on the overall pipeline. Ultimately, the best standalone model is identified—this model is not necessarily selected for the cascade system, as peak standalone performance may not translate to optimal combined performance. The predictions from all intermediate models during this phase are used to simulate the cascade in the subsequent experiment detailed in Section 4.2.2.

1. **Preprocessing and basic representation.** Using the default `CountVectorizer` with $(1, 3)$ n-grams, the four combinations of text variant (clean vs. expanded) and casing (cased vs. lowercased) are evaluated with both BoW and TF-IDF for the linear SVM and LR. This stage identifies the best preprocessing choice (expanded, lowercased, TF-IDF).

2. **Custom vectorization and Z-score pruning.** All six token representations (cased, lower, letters, filtered, stemmed, lemmatized) are combined with Z-score thresholds $Z \in \{0, 1, 2, 3, 4\}$ and both BoW and TF-IDF, using the custom space-tokenizer described in Section 3.3.1. The best token representation and pruning level are chosen for each model.

3. **Vocabulary depth (n-gram range).** With the best token representation and $Z = 2.0$ fixed, the n-gram range is varied from $(1, 1)$ to $(1, 3)$. This test measures the trade-off between accuracy and feature-space size.

4. **Kernel type.** With the same settings as in the previous step, the impact of the kernel is assessed. The accuracy and training times of SVMs with a linear kernel and an RBF kernel are compared at two `max_iter` limits: $1,000$ and $10,000$.

5. **Semantic clustering.** For $k \in \{500, 5\,000, 10\,000\}$ and sentiment weights $w \in \{0, 1, 10\}$, concept matrices are built from the optimal unit features (Section 3.3.3)

and evaluated with both SVM and LR. This gauges whether concept compression can match the unclustered baseline.

6. **Cascade threshold sweep.** For every candidate base model from the configuration enumeration (Section 3.3.5), a sweep over $\tau \in [0, 0.5]$ is performed on the validation set, simulating the cascade. The pair (base model, τ) that maximizes the hybrid accuracy is selected, and identifying the optimal base model (Section 3.4.2).

4.2.2 Cascade System Experiment

The cascade architecture depends on the assumption that the base model's predictive confidence correlates with correctness, so that delegating uncertain instances to a specialist can improve overall performance. The experiments in this subsection first validate this confidence assumption. Then, three alternative specialist strategies are compared to determine whether a computationally heavy model is required, or whether the same model type trained on the boundary region is sufficient.

Confidence Analysis

To verify that the delegation threshold τ genuinely isolates difficult cases, the base model's predicted probabilities are binned and the empirical accuracy in each bin is examined. For instances with probabilities close to 0 or 1 (the acceptance regions), the observed accuracy and coverage should be substantially higher than for instances with probabilities near 0.5 (the boundary region) to confirm that the interval $[\tau, 1 - \tau]$ captures cases where the base model is most error-prone.

Specialist Comparison

Three cascade variants are evaluated to assess which specialist configuration yields the best performance. The base model is fixed to the optimal linear SVM identified by the sweep (Section 3.4.2), and the delegation threshold is set to $\tau = 0.254$ (Section 3.4.2). Three specialist variants are tested:

- **Base DistilBERT**: the generic DistilBERT model (Section 3.3.4).
- **SVM specialist**: the linear SVM introduced in Section 3.4.3.
- **BERT specialist**: the fine-tuned DistilBERT described in Section 3.4.3.

All three cascade variants are evaluated on the held-out test set, together with the standalone base SVM and the standalone DistilBERT baseline. The final cascade prediction follows the combination rule defined in Section 3.4.1: the base model's output is used for instances outside the boundary region, and the specialist's output is used for instances inside it.

4.3 Evaluation Metrics

Because the data splits are exactly balanced through stratified sampling, accuracy and macro F_1-score are equally reliable indicators of performance. Both are reported in all experiments. The primary metric for selecting the best configuration is accuracy, as it directly aligns with the McNemar test.

4.3.1 Notation

Let N be the number of test instances. For each instance $\iota \in \{1, \ldots, N\}$, let $y_\iota \in \{0, 1\}$ denote the true label and $\hat{y}_\iota \in \{0, 1\}$ the predicted label.

Accuracy is the fraction of correct predictions:

$$\text{Accuracy} = \frac{1}{N} \sum_{\iota=1}^{N} \mathbf{1}[\hat{y}_\iota = y_\iota], \tag{4.1}$$

Macro $\mathbf{F_1}$-score averages the F_1 score over the two classes without weighting. For a given class $c \in \{0, 1\}$, let TP_c, FP_c, FN_c be the true positives, false positives, and false negatives, respectively. The per-class precision, recall, and F_1 are:

$$P_c = \frac{\text{TP}_c}{\text{TP}_c + \text{FP}_c} \tag{4.2}$$

$$R_c = \frac{\text{TP}_c}{\text{TP}_c + \text{FN}_c} \tag{4.3}$$

$$F_{1,c} = 2 \cdot \frac{P_c \cdot R_c}{P_c + R_c} \tag{4.4}$$

The macro F_1-score is the arithmetic mean of the two per-class values:

$$\text{Macro } F_1 = \frac{1}{2} \sum_{c=0}^{1} F_{1,c} \tag{4.5}$$

4.3.2 Statistical Significance Testing

All pairwise model comparisons are validated with McNemar's test at the $p = 0.05$ significance level. The test evaluates whether the error distributions of two classifiers, computed on the same set of instances, differ significantly. In each experiment a baseline is chosen—typically the best-performing configuration from the previous stage. The implementation follows `statsmodels`, using the chi-squared approximation with a continuity correction (`exact=False, correction=True`). Raw p-values are reported without multiplicity correction, given the sequential, hypothesis-driven design.

Let n_{10} be the number of instances misclassified by the baseline only, and n_{01} the number misclassified by the challenger only. The null hypothesis is $H_0 : n_{10} = n_{01}$ (the two classifiers have the same error distribution). The test statistic is:

$$\chi^2 = \frac{(|n_{10} - n_{01}| - 1)^2}{n_{10} + n_{01}}, \tag{4.6}$$

which, under H_0, approximately follows a chi-squared (χ^2) distribution with one degree of freedom.

The p-value is the probability that a χ^2_1 random variable would be at least as large as the observed value χ^2_{obs}. In the code, this is computed via the survival function: `stats.chi2.sf(`χ^2`, df=1)`. If $p < 0.05$, the difference between the two classifiers is considered statistically significant. Because the statistic squares the absolute difference, the test is two-sided: it detects any deviation from H_0, irrespective of increase or decrease in the accuracy.

4.3.3 Visualization of Statistical Significance

Every figure and table consistently encodes the outcome of McNemar's test:

- **Graphs.** Significant comparisons ($p < 0.05$) are drawn with full colour saturation, thick lines, and solid markers; non-significant ones use low opacity, thinner lines, and hollow markers. The legend carries the tags "Sig. Diff" and "Not Sig." where appropriate.

- **Tabular reports.** Rows with $p < 0.05$ are set in bold and accompanied, when a significant result is obtained, a directional arrow (↑ for improvement, ↓ for degradation) is added based on the raw accuracy difference. The baseline row is also bold. Rows with $p \geq 0.05$ use regular weight and no arrow, as illustrated in Table 4.1.

Table 4.1: Example results of the McNemar test. Bold text indicates statistical significance, while directional arrows denote a significant increase or decrease in accuracy of the challenger model relative to the baseline.

Model Description	Acc_{Model}	Direction	p-value
Challenger A (significantly better)	**0.8542**	↑	**0.001**
Baseline model	**0.8310**		—
Challenger B (significantly worse)	**0.8103**	↓	**0.023**
Challenger C (insignificant)	0.8325		0.4122

4.4 Results

This section presents the evaluation of linear SVM and LR classifiers across various configurations. Initial tests establish that linear SVMs consistently outperform LR models, with TF-IDF representations universally yielding superior predictive performance compared to standard BoW. The findings demonstrate that implementing statistical pruning effectively mitigates the dimensionality explosion associated with complex n-grams while preserving optimal classification accuracy. Ultimately, the results validate the proposed cascade ensemble methodology; delegating uncertain predictions from a highly efficient linear base model to a deep-semantic BERT specialist resolves non-linear contextual cues, achieving a peak combined macro F_1-score of approximately 92.3% and significantly outperforming standalone baselines.

The analysis is split into several stages, each describing an individual impact:

1. **Experimental Evaluation Sequence:**
    - Impact of Casing and Text Expansion (Sec. 4.4.1)
    - Impact of Filtering (Sec. 4.4.2)
    - Impact of Custom Vectorization (Sec. 4.4.3)
    - Impact of Vocabulary Depth (Sec. 4.4.4)
    - Impact of Kernel (Sec. 4.4.5)
    - Impact of Semantic Clustering (Sec. 4.4.6)
2. **Cascade System Experiment:**
    - Impact of Certainty (Sec. 4.4.7)
    - Impact of Cascade Ensemble (Sec. 4.4.8)

4.4.1 Impact of Casing and Text Expansion

In this test, the performance of two classical classifiers—linear SVM and LR—is evaluated across varying preprocessing and feature representation configurations. All models were trained on the training set. The evaluation metrics (macro F_1-score and accuracy) are computed on the validation set. The experimental conditions are categorized as follows:

- Minimal preprocessing (Clean) vs. Contraction expansion (Expanded),
- Original capitalization (Cased) vs. Lowercased (Lower),
- BoW vs. TF-IDF (`ngram_range=(1,3)`, `min_df=10`, `max_df=0.7`).

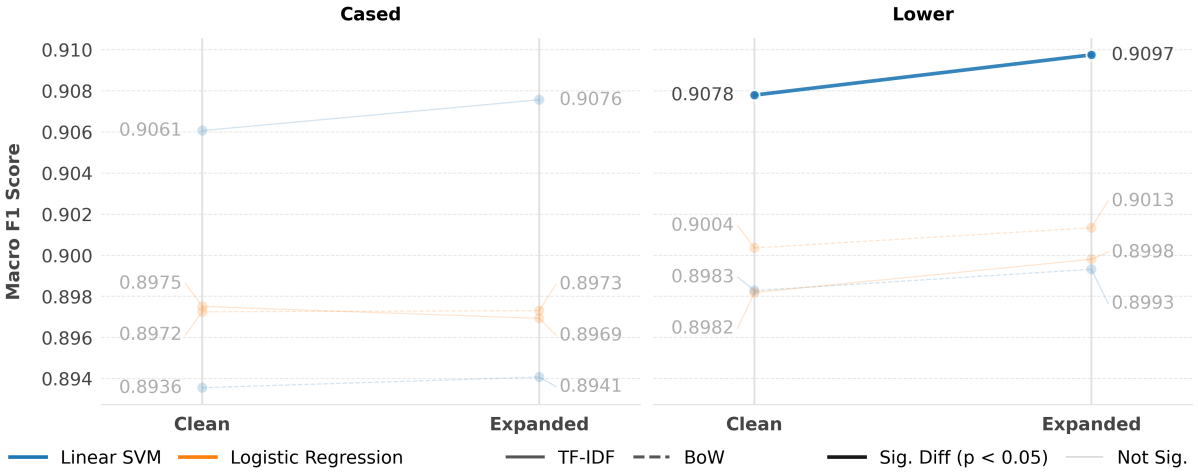


Figure 4.1: Macro F_1-scores comparing clean versus expanded text. The left and right panels display results for cased and lowercased text, respectively. Line colors differentiate the machine learning models (blue for Linear SVM, orange for Logistic Regression), while line styles denote the text representation (solid for TF-IDF, dashed for BoW). Full line opacity indicates a statistically significant difference ($p < 0.05$), with faded lines denoting insignificant changes. Only the lowercased Linear SVM utilizing TF-IDF demonstrated a significant performance impact upon expansion.

As illustrated in Figure 4.1, the transition from cleaned to expanded text provides a statistically significant benefit for lower-cased text, as demonstrated by McNemar's test applied to the pairwise predictions. Other transitions are not found to be statistically significant and are thus grayed out on the graph.

Comparative Analysis of Configurations

Tables 4.2a through 4.2d provide a cross-configuration analysis, comparing various models and representations against selected baselines. The first two tables use SVMs as the baseline models, and the latter two use LR. From these results, several key trends emerge:

1. **Model Impact:** linear SVM consistently outperforms LR models across all equivalent settings.

2. **Representation Impact:** TF-IDF features yield better performance than BoW.

Table 4.2b and Figure 4.1 highlight that a performance gain occurs when expansion is applied to lowercased text—`Expanded and Lower` configuration achieves the highest accuracy (90.98%), and proves significantly better than the standard "Lower" baseline.

4.4.2 Impact of Filtering

POS filtering reduces the overall dimensionality of the feature space by approximately 17%. For example, the number of positive tokens decreased from 19,172 to 15,794, and the

Table 4.2: McNemar test results analyzing the impact of casing, text expansion, representation, and model choice. Across all subtables, bold text indicates a statistically significant difference ($p < 0.05$) relative to the baseline, with arrows denoting the direction of the change in accuracy.

(a) Baseline: SVM with TF-IDF on cased text. Evaluated variations: model (LR), representation (BoW), and text lower-casing/expansion.

	Text	Representation	Model	Acc_{Model}	↑/↓	p-value
0	**Cased**	**TF-IDF**	**SVM**	**90.61%**		**Baseline**
1	**Cased**	**TF-IDF**	**LR**	**89.75%**	↓	**0.000000**
2	**Cased**	**BoW**	**SVM**	**89.36%**	↓	**0.000000**
3	Lower	TF-IDF	SVM	90.78%		0.164879
4	Expanded and Cased	TF-IDF	SVM	90.76%		0.112399

(b) Baseline: SVM with TF-IDF on lowered text. Evaluated variations: model (LR), representation (BoW), and text casing/expansion.

	Text	Representation	Model	Acc_{Model}	↑/↓	p-value
0	**Expanded and Lower**	**TF-IDF**	**SVM**	**90.98%**	↑	**0.036879**
1	**Lower**	**TF-IDF**	**SVM**	**90.78%**		**Baseline**
2	**Lower**	**BoW**	**SVM**	**89.83%**	↓	**0.000000**
3	**Lower**	**TF-IDF**	**LR**	**89.82%**	↓	**0.000000**
4	Cased	TF-IDF	SVM	90.61%		0.164879

(c) Baseline: LR with TF-IDF on cased text. Evaluated variations: model (SVM), representation (BoW), and text lower-casing/expansion.

	Text	Representation	Model	Acc_{Model}	↑/↓	p-value
0	**Cased**	**TF-IDF**	**SVM**	**90.61%**	↑	**0.000000**
1	**Cased**	**TF-IDF**	**LR**	**89.75%**		**Baseline**
2	Lower	TF-IDF	LR	89.82%		0.546041
3	Cased	BoW	LR	89.73%		0.898788
4	Expanded and Cased	TF-IDF	LR	89.70%		0.513802

(d) Baseline: LR with TF-IDF on lowered text. Evaluated variations: model (SVM), representation (BoW), and text casing/expansion.

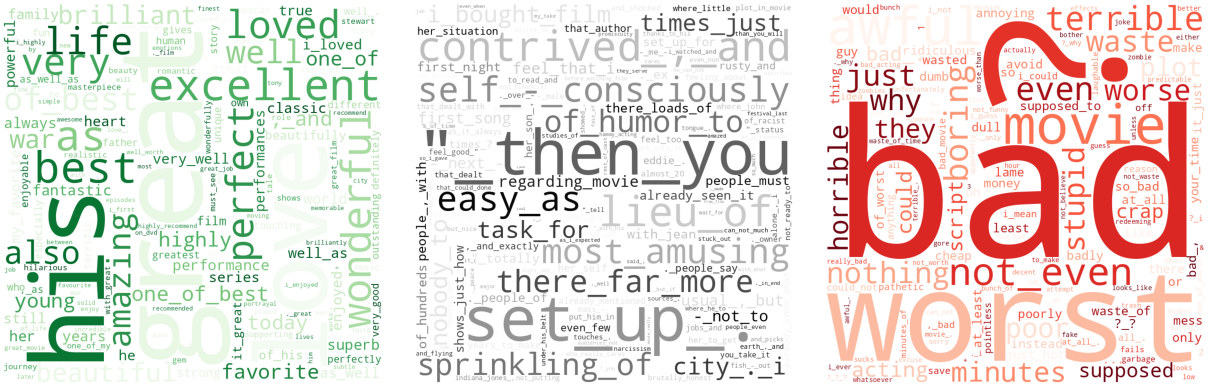
	Text	Representation	Model	Acc_{Model}	↑/↓	p-value
0	**Lower**	**TF-IDF**	**SVM**	**90.78%**	↑	**0.000000**
1	**Lower**	**TF-IDF**	**LR**	**89.82%**		**Baseline**
2	Lower	Bow	LR	90.04%		0.242561
3	Expanded and Lower	TF-IDF	LR	89.98%		0.053784
4	Cased	TF-IDF	LR	89.75%		0.546041

(a) Extracted positive words (total kept: 19172)

(b) Extracted neutral words (total discarded: 102665)

(c) Extracted negative words (total kept: 17178)

Figure 4.2: Word clouds for `tokens_lower` (kept and discarded by Z-score).



(a) Extracted positive words (total kept: 15794)

(b) Extracted neutral words (total discarded: 85174)

(c) Extracted negative words (total kept: 14268)

Figure 4.3: Word clouds for `tokens_filtered` (kept and discarded by Z-score).

volume of discarded neutral tokens dropped from 102,665 to 85,174. In the word clouds (Figure 4.2 and Figure 4.3), the size of each word is proportional to its Z-score, which represents its correlation with the sentiment. By discarding neutral, the filtered approach amplifies the polarity of key sentiment indicators. For example, discriminative words such as *"worst"* achieve notably higher Z-scores in the filtered configuration. Consequently, they appear visibly larger in Figure 4.3.

Observations

It is worth noting that the results are easily interpretable. Tokens such as *"great"* or *"love"* indicate positive sentiment, while *"bad"*, *"worst"*, or *"boring"* indicate negative feelings towards a movie. It is also interesting that *"his"*, *"episodes"*, or *"music"* appear in the positive group, suggesting that people tend to be more descriptive when they write a positive review. Tokens such as *"?"*, *"why"*, and *"..."* appear in the negative reviews.

4.4.3 Impact of Custom Vectorization

Instead of relying on the default `CountVectorizer`, the standard regular expression tokenization is bypassed. After the vectorization, neutral tokens are pruned based on their calculated Z-scores resulting in Custom BoW (CBoW) and Custom TF-IDF (CTf-IDF).

The following test evaluates various tokenization representations to identify the optimal Z-score pruning threshold, utilizing the best-performing model from the previous stage as a baseline. As illustrated in Figure 4.4, the models' macro F_1-scores can remain highly stable even when pruning is applied. However, the effectiveness of pruning is dependent on the chosen feature representation. For the linear SVM utilizing a CBoW, statistical pruning proves not applicable; performance degrades immediately, making $Z = 0$ (no pruning) the optimal threshold. For the LR model with CBoW, the performance remains stable up to a threshold of $Z = 1$ before declining.

A particularly interesting pattern emerges when utilizing the CTf-IDF representation. For both the linear SVM and LR models, the optimal pruning threshold shifts significantly to $Z = 2$. At this threshold, the SVM maintains its peak baseline performance despite a substantial reduction in the feature space. Notably, the LR model exhibits a slight increase in its F_1-score, suggesting that the targeted removal of neutral features actively improves the model's ability to generalize.

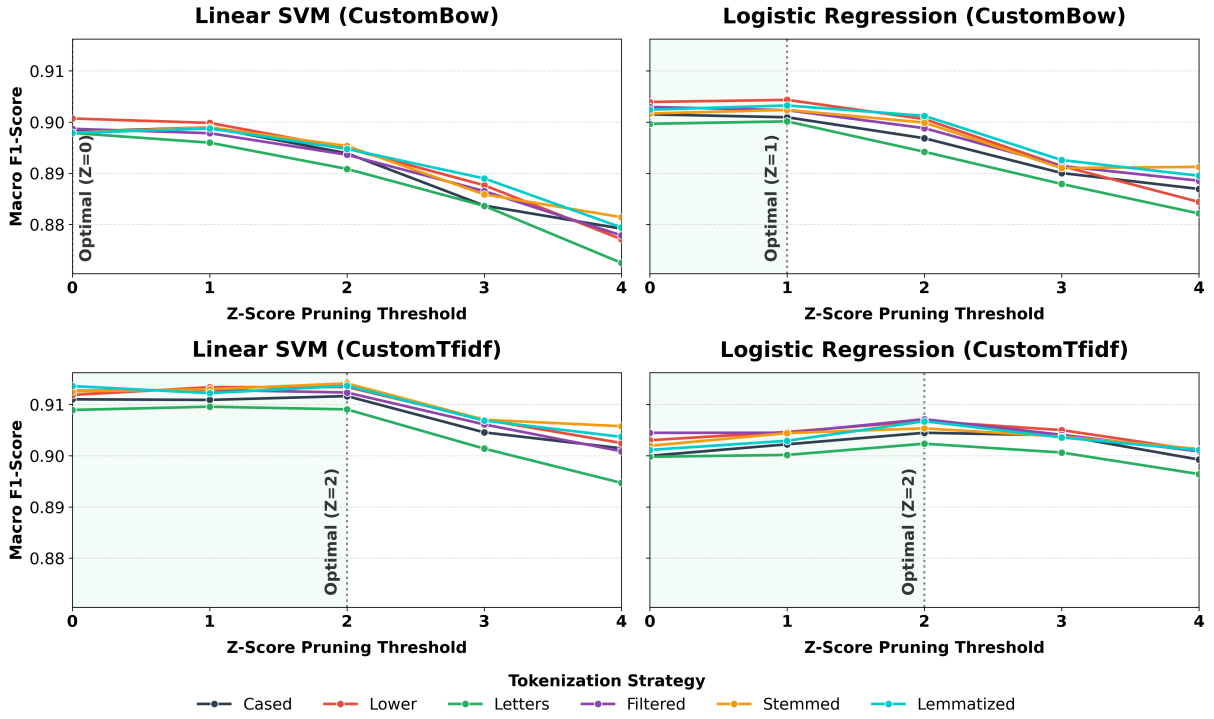


Figure 4.4: Impact of neutral vocabulary pruning on macro F_1-score. The 2×2 grid compares CustomBow (top row) and CustomTfidf (bottom row) representations across SVM (left column) and LR (right column) models. Line colors distinguish the six tokenization strategies. Vertical dotted lines annotate the optimal pruning threshold, with the shaded green regions highlighting the acceptable threshold range prior to performance loss.

Impact of Token Type

As detailed in Table 4.3, the most effective token representations rely on stemming, lemmatization, and simple lowercasing. The optimal configuration utilizes stemmed tokens paired with a Z-score pruning threshold of $Z = 2$, achieving a peak accuracy of 91.41% ($p = 0.0125$). Lemmatization/Stemming collapses tokens to their morphological roots and thus reduces the feature dimensionality compared to standard tokenization. Additionally, the lemmatized representations at pruning thresholds $Z = 0$ and $Z = 2$ achieve a nearly identical accuracy—reducing the dimensionality even more—this shows that pruning can be employed without negatively affecting the accuracy.

While a few configurations at $Z = 1$ and $Z = 2$ yield improvements over the baseline ($\uparrow$), the vast majority of combinations between $Z = 0$ and $Z = 3$ result in p-values greater than 0.05. This indicates that for most representation strategies, moderate statistical pruning perfectly maintains classification performance on par with the unpruned baseline. Significant degradation ($\downarrow$) only begins to manifest under aggressive pruning at $Z = 4$.

As detailed in Table 4.4, the simplest representation (Tokens Letters with $Z = 0$) is chosen as the baseline. Compared to this baseline, a statistically significant increase in accuracy is observed when utilizing more sophisticated representations.

Notably, the filtered representation combined with $Z = 2$ pruning achieves the highest overall accuracy for the LR model (90.72%). This contrasts with the earlier findings for the linear SVM, emphasizing that feature selection strategies are highly model-dependent.

Impact of Representation

The CTf-IDF representation achieves higher accuracy than the `CBoW` approach across all tested configurations (see Table 4.5). This outcome verifies that the pattern observed in the initial tests (Tables 4.2a through 4.2d) holds true: TF-IDF outperforms BoW for the sentiment analysis task. The superiority of TF-IDF is robust and largely independent of the tokenization strategy. Additionally, consistent with the trends previously illustrated in Figure 4.4, aggressive pruning at $Z = 4$ results in performance degradation.

Ablation Study

In Figure 4.5, the BoW and TF-IDF labels denote the unpruned token representations. Meanwhile, CBoW and CTf-IDF designate where pruning ($Z = 2$) has been applied.

For the BoW, various tokenization strategies yield no improvement over the `Expanded and Lower` baseline for either the linear SVM or LR models. Furthermore, when statistical pruning is introduced (CBoW), performance consistently degrades across all configurations; removing vocabulary features universally harms the predictive power of BoW.

In contrast, the TF-IDF representations exhibit a lot of variance. For the linear SVM, only the lemmatized tokenization approach proves significantly better than the TF-IDF

Table 4.3: McNemar test results comparing the baseline TF-IDF representation against custom (CTf-IDF) representations with different Z-score pruning values. Only SVM models are considered. Bold text indicates a statistically significant difference ($p < 0.05$) relative to the baseline, with arrows denoting the direction of the change in accuracy.

	Text	Representation	Model	Acc_{Model}	↑/↓	p-value
0	**Tokens Stemmed**	**CTf-IDF (Z2)**	**SVM**	**91.41%**	↑	**0.012517**
1	**Tokens Lemmatized**	**CTf-IDF (Z2)**	**SVM**	**91.37%**	↑	**0.020340**
2	**Tokens Lemmatized**	**CTf-IDF (Z0)**	**SVM**	**91.36%**	↑	**0.010486**
3	**Tokens Lower**	**CTf-IDF (Z2)**	**SVM**	**91.34%**	↑	**0.026857**
4	**Tokens Lower**	**CTf-IDF (Z1)**	**SVM**	**91.34%**	↑	**0.018008**
5	**Tokens Stemmed**	**CTf-IDF (Z1)**	**SVM**	**91.30%**	↑	**0.049788**
6	**Expanded and Lower**	**TF-IDF**	**SVM**	**90.98%**		**Baseline**
7	**Tokens Stemmed**	**CTf-IDF (Z4)**	**SVM**	**90.58%**	↓	**0.047471**
8	**Tokens Cased**	**CTf-IDF (Z3)**	**SVM**	**90.46%**	↓	**0.006363**
9	**Tokens Lemmatized**	**CTf-IDF (Z4)**	**SVM**	**90.37%**	↓	**0.002455**
10	**Tokens Lower**	**CTf-IDF (Z4)**	**SVM**	**90.25%**	↓	**0.000225**
11	**Tokens Cased**	**CTf-IDF (Z4)**	**SVM**	**90.15%**	↓	**0.000044**
12	**Tokens Letters**	**CTf-IDF (Z3)**	**SVM**	**90.14%**	↓	**0.000005**
13	**Tokens Filtered**	**CTf-IDF (Z4)**	**SVM**	**90.09%**	↓	**0.000013**
14	**Tokens Letters**	**CTf-IDF (Z4)**	**SVM**	**89.47%**	↓	**0.000000**
15	Tokens Filtered	CTf-IDF (Z1)	SVM	91.29%		0.059020
16	Tokens Filtered	CTf-IDF (Z0)	SVM	91.27%		0.065874
17	Tokens Stemmed	CTf-IDF (Z0)	SVM	91.26%		0.075040
18	Tokens Filtered	CTf-IDF (Z2)	SVM	91.23%		0.147101
19	Tokens Lemmatized	CTf-IDF (Z1)	SVM	91.22%		0.119306
20	Tokens Lower	CTf-IDF (Z0)	SVM	91.19%		0.141313
21	Tokens Cased	CTf-IDF (Z2)	SVM	91.17%		0.287989
22	Tokens Cased	CTf-IDF (Z0)	SVM	91.10%		0.434486
23	Tokens Cased	CTf-IDF (Z1)	SVM	91.09%		0.500669
24	Tokens Letters	CTf-IDF (Z1)	SVM	90.96%		0.929637
25	Tokens Letters	CTf-IDF (Z2)	SVM	90.91%		0.681427
26	Tokens Letters	CTf-IDF (Z0)	SVM	90.89%		0.490824
27	Tokens Stemmed	CTf-IDF (Z3)	SVM	90.70%		0.161398
28	Tokens Lower	CTf-IDF (Z3)	SVM	90.70%		0.147175
29	Tokens Lemmatized	CTf-IDF (Z3)	SVM	90.69%		0.125708
30	Tokens Filtered	CTf-IDF (Z3)	SVM	90.61%		0.060745

Table 4.4: McNemar test results comparing the simplest representation baseline (alphabetical filtering, CTf-IDF with no pruning) against custom representations with different Z-score pruning values. Only LR models are considered. Bold text indicates a statistically significant difference ($p < 0.05$) relative to the baseline, with arrows denoting the direction of the change in accuracy.

	Text	Representation	Model	Acc_{Model}	↑/↓	p-value
0	**Tokens Filtered**	**CTf-IDF (Z2)**	**LR**	**90.72%**	↑	**0.000007**
1	**Tokens Lower**	**CTf-IDF (Z2)**	**LR**	**90.68%**	↑	**0.000003**
2	**Tokens Lemmatized**	**CTf-IDF (Z2)**	**LR**	**90.68%**	↑	**0.000006**
3	**Tokens Stemmed**	**CTf-IDF (Z2)**	**LR**	**90.54%**	↑	**0.000541**
4	**Tokens Lower**	**CTf-IDF (Z3)**	**LR**	**90.50%**	↑	**0.001052**
5	**Tokens Lower**	**CTf-IDF (Z1)**	**LR**	**90.47%**	↑	**0.000508**
6	**Tokens Filtered**	**CTf-IDF (Z1)**	**LR**	**90.45%**	↑	**0.003916**
7	**Tokens Cased**	**CTf-IDF (Z2)**	**LR**	**90.45%**	↑	**0.002850**
8	**Tokens Filtered**	**CTf-IDF (Z0)**	**LR**	**90.45%**	↑	**0.003823**
9	**Tokens Stemmed**	**CTf-IDF (Z1)**	**LR**	**90.45%**	↑	**0.002138**
10	**Tokens Filtered**	**CTf-IDF (Z3)**	**LR**	**90.41%**	↑	**0.014078**
11	**Tokens Cased**	**CTf-IDF (Z3)**	**LR**	**90.40%**	↑	**0.013048**
12	**Tokens Stemmed**	**CTf-IDF (Z3)**	**LR**	**90.38%**	↑	**0.020793**
13	**Tokens Lemmatized**	**CTf-IDF (Z3)**	**LR**	**90.36%**	↑	**0.020176**
14	**Tokens Lower**	**CTf-IDF (Z0)**	**LR**	**90.31%**	↑	**0.017309**
15	**Tokens Lemmatized**	**CTf-IDF (Z1)**	**LR**	**90.30%**	↑	**0.035011**
16	**Tokens Letters**	**CTf-IDF (Z0)**	**LR**	**89.98%**		**Baseline**
17	**Tokens Letters**	**CTf-IDF (Z4)**	**LR**	**89.64%**	↓	**0.035752**
18	Tokens Letters	CTf-IDF (Z2)	LR	90.24%		0.053513
19	Tokens Cased	CTf-IDF (Z1)	LR	90.23%		0.108343
20	Tokens Stemmed	CTf-IDF (Z0)	LR	90.20%		0.163340
21	Tokens Stemmed	CTf-IDF (Z4)	LR	90.13%		0.410827
22	Tokens Lemmatized	CTf-IDF (Z0)	LR	90.12%		0.375011
23	Tokens Lemmatized	CTf-IDF (Z4)	LR	90.11%		0.477496
24	Tokens Lower	CTf-IDF (Z4)	LR	90.11%		0.492971
25	Tokens Filtered	CTf-IDF (Z4)	LR	90.09%		0.588993
26	Tokens Letters	CTf-IDF (Z3)	LR	90.07%		0.605078
27	Tokens Letters	CTf-IDF (Z1)	LR	90.02%		0.785028
28	Tokens Cased	CTf-IDF (Z0)	LR	90.00%		0.937425
29	Tokens Cased	CTf-IDF (Z4)	LR	89.93%		0.766185

Table 4.5: McNemar test results comparing the baseline SVM model (lowered tokens, CTf-IDF, $Z = 2$) against Custom Bag-of-Words (CBoW) representations. Only SVM models are considered. Bold text indicates a statistically significant difference ($p < 0.05$) relative to the baseline, with arrows denoting the direction of the change in accuracy.

	Text	Representation	Model	Acc_{Model}	↑/↓	p-value
0	**Tokens Lower**	**CTf-IDF (Z2)**	**SVM**	**91.34%**		**Baseline**
1	**Tokens Lower**	**CBoW (Z0)**	**SVM**	**90.07%**	↓	**0.000000**
2	**Tokens Lower**	**CBoW (Z1)**	**SVM**	**89.98%**	↓	**0.000000**
3	**Tokens Cased**	**CBoW (Z1)**	**SVM**	**89.90%**	↓	**0.000000**
4	**Tokens Stemmed**	**CBoW (Z1)**	**SVM**	**89.90%**	↓	**0.000000**
5	**Tokens Lemmatized**	**CBoW (Z1)**	**SVM**	**89.87%**	↓	**0.000000**
6	**Tokens Filtered**	**CBoW (Z0)**	**SVM**	**89.87%**	↓	**0.000000**
7	**Tokens Cased**	**CBoW (Z0)**	**SVM**	**89.81%**	↓	**0.000000**
8	**Tokens Stemmed**	**CBoW (Z0)**	**SVM**	**89.79%**	↓	**0.000000**
9	**Tokens Lemmatized**	**CBoW (Z0)**	**SVM**	**89.79%**	↓	**0.000000**
10	**Tokens Letters**	**CBoW (Z0)**	**SVM**	**89.79%**	↓	**0.000000**
11	**Tokens Filtered**	**CBoW (Z1)**	**SVM**	**89.78%**	↓	**0.000000**
12	**Tokens Letters**	**CBoW (Z1)**	**SVM**	**89.60%**	↓	**0.000000**
13	**Tokens Stemmed**	**CBoW (Z2)**	**SVM**	**89.53%**	↓	**0.000000**
14	**Tokens Lower**	**CBoW (Z2)**	**SVM**	**89.49%**	↓	**0.000000**
15	**Tokens Lemmatized**	**CBoW (Z2)**	**SVM**	**89.48%**	↓	**0.000000**
16	**Tokens Cased**	**CBoW (Z2)**	**SVM**	**89.39%**	↓	**0.000000**
17	**Tokens Filtered**	**CBoW (Z2)**	**SVM**	**89.36%**	↓	**0.000000**
18	**Tokens Letters**	**CBoW (Z2)**	**SVM**	**89.09%**	↓	**0.000000**
19	**Tokens Lemmatized**	**CBoW (Z3)**	**SVM**	**88.90%**	↓	**0.000000**
20	**Tokens Lower**	**CBoW (Z3)**	**SVM**	**88.77%**	↓	**0.000000**
21	**Tokens Filtered**	**CBoW (Z3)**	**SVM**	**88.65%**	↓	**0.000000**
22	**Tokens Stemmed**	**CBoW (Z3)**	**SVM**	**88.59%**	↓	**0.000000**
23	**Tokens Cased**	**CBoW (Z3)**	**SVM**	**88.37%**	↓	**0.000000**
24	**Tokens Letters**	**CBoW (Z3)**	**SVM**	**88.37%**	↓	**0.000000**
25	**Tokens Stemmed**	**CBoW (Z4)**	**SVM**	**88.15%**	↓	**0.000000**
26	**Tokens Lemmatized**	**CBoW (Z4)**	**SVM**	**87.94%**	↓	**0.000000**
27	**Tokens Cased**	**CBoW (Z4)**	**SVM**	**87.92%**	↓	**0.000000**
28	**Tokens Filtered**	**CBoW (Z4)**	**SVM**	**87.79%**	↓	**0.000000**
29	**Tokens Lower**	**CBoW (Z4)**	**SVM**	**87.71%**	↓	**0.000000**
30	**Tokens Letters**	**CBoW (Z4)**	**SVM**	**87.25%**	↓	**0.000000**

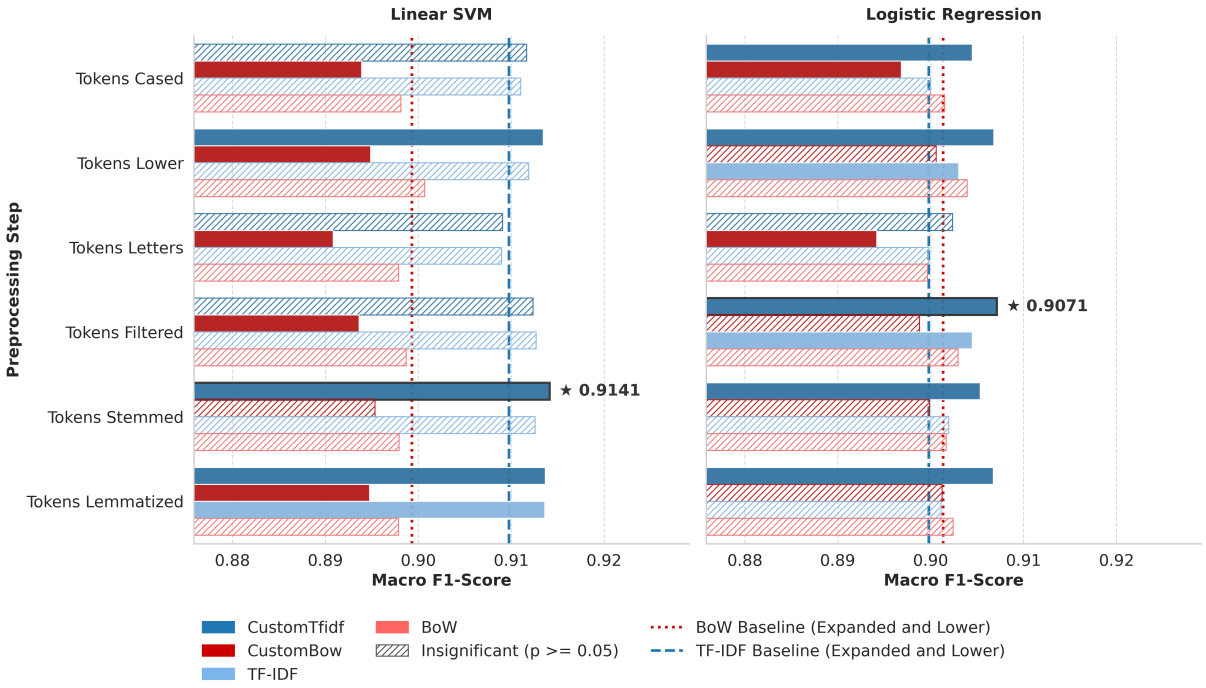


Figure 4.5: Ablation study evaluating standard tokenized representations versus custom representations with statistical pruning across various preprocessing steps. Bar colors distinguish the representation family (blues for TF-IDF, reds for BoW), vertical lines mark the baseline performance on lower-cased expanded text. Hashed bars indicate results that are not statistically significantly different from their respective baseline ($p \geq 0.05$). The top-performing configurations for each model are outlined and annotated with a star.

Baseline. However, the introduction of $Z = 2$ pruning actively enhances the model's capabilities, increasing the macro F_1-scores for the lowered and stemmed representations. Combining stemming and CTf-IDF achieves the highest overall performance (91.41%).

For the LR model, the classification scores are generally lower than those achieved by the SVM. For the BoW approach, pruning either degrades performance (as seen with cased and letter-only tokens) or has no significant effect. However, the introduction of filtering combined with CTf-IDF pruning proves effective. While isolated tokenization approaches do not have a clear impact on their own, the pruning strategy elevates almost all TF-IDF-based representations for LR, with the sole exception of the `Tokens Letters` configuration. The `Tokens Filtered` representation with $Z = 2$ pruning outperforms the baseline by over 0.7 percentage points (rising from 89.98% to 90.71%).

4.4.4 Impact of Vocabulary Depth

This test examines the importance of selecting the optimal n-gram range—the vocabulary depth. As depicted in the left panel of Figure 4.6, expanding the token representation from simple unigrams $(1, 1)$ to include bigrams $(1, 2)$ yields a substantial increase in the macro F_1-score for both the linear SVM and LR models. However, extending the depth

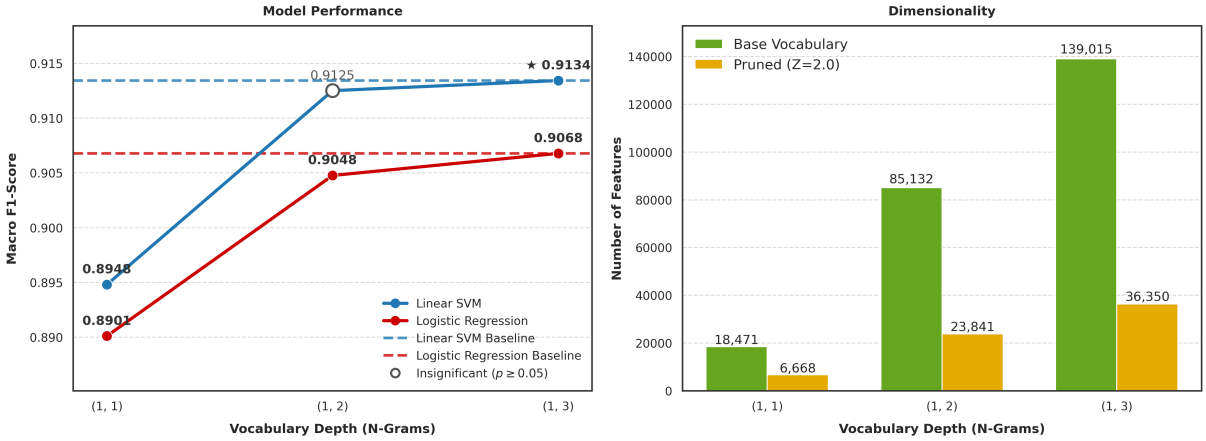


Figure 4.6: Performance and dimensionality trends across varying n-gram ranges. The left panel illustrates model performance (macro F_1-score), where dashed lines indicate the respective baselines for Linear SVM (blue) and LR (red). Hollow markers denote results that are not statistically significantly different from their baseline ($p \geq 0.05$), and a star highlights the overall best-performing configuration. The right panel highlights the feature space reduction, comparing the base vocabulary size (green bars) against the pruned vocabulary using $Z = 2.0$ (orange bars).

further to include trigrams $(1, 3)$ provides only marginal performance gains, illustrating a clear pattern of diminishing returns.

Dimensionality Problem

While deeper n-gram ranges capture richer sequential context, they introduce a severe structural challenge: the explosion of the feature space. The right panel of Figure 4.6 demonstrates that the base vocabulary grows aggressively, swelling from 18,471 unigrams to 139,015 total features when trigrams are included. Extrapolating this trajectory suggests that exploring even deeper ranges, such as 4-grams $(1, 4)$, would predictably push the raw feature count well beyond 190,000. Operating on such massive, highly sparse matrices becomes extremely memory-intensive.

Dimensionality Reduction

The implementation of statistical pruning ($Z = 2.0$) effectively mitigates this dimensionality explosion. For the $(1, 3)$ vocabulary range, pruning discards over 100,000 non-discriminative features, compressing the space from 139,015 down to manageable 36,350 features. This ensures that the classifiers can leverage the contextual richness of higher-order n-grams without suffering the standard computational penalties associated with high dimensionality.

Table 4.6: McNemar's test comparing linear and RBF kernel SVMs, including training times measured on `AMD Ryzen 7 5800H with Radeon Graphics` at different iteration limits (`max_iter`).

	`max_iter`	Training Time	Model	Acc_{Model}	↑/↓	p-value
0	**10,000**	**1.89 s**	**Linear SVM**	**91.34%**		**Baseline**
1	**1,000**	**267.07 s**	**RBF SVM**	**89.03%**	↓	**0.000000**
2	10,000	2042.05 s	RBF SVM	91.15%		0.142368

4.4.5 Impact of Kernel

To assess the influence of the kernel function, a linear SVM is compared against an RBF SVM, using the best previously determined text representation and preprocessing settings (see Table 4.6). Both models were trained on the full training set, with probability calibration performed internally via 5-fold cross-validation (`CalibratedClassifierCV`)—no cross-validation accuracy metrics are exported by the calibration procedure.

Both variants use a fixed `max_iter` limit. The linear SVM with `max_iter=10000` trains and converges in only 1.89 seconds, achieving 91.34% accuracy. In contrast, the RBF kernel with the same iteration limit trains for 2042.05 seconds (over 34 minutes) and reaches 91.15% accuracy. However, the RBF solver did not converge: it exhausted the maximum number of iterations before reaching the stopping criterion. This indicates that the RBF kernel's optimization landscape is harder to navigate, and requires more time for proper convergence.

When `max_iter` is reduced to 1,000, the RBF training time drops to 267.07 seconds (approximately 4.5 minutes), but accuracy falls to 89.03%. The difference between this RBF variant and the linear baseline is significant according to McNemar's test ($p < 0.000001$), whereas the non-converged RBF with 10,000 iterations does not exhibit a statistically significant difference from the linear SVM ($p = 0.142$). Overall, the linear kernel yields the highest accuracy and has significantly lower computational cost, making it preferable for the further tests.

4.4.6 Impact of Semantic Clustering

Semantic clustering groups related n-grams to capture conceptual representations. As shown in Figure 4.7, the cluster containing *"great"*, *"wonderful"*, and *"amazing"* forms the most discriminative positive concept, achieving a Z-score of 44.20. Clusters focused on structural phrases, such as *"his"* or *"is an"* rank among the top positive concepts, reinforcing the earlier observation that reviewers tend to use more descriptive language when expressing a positive opinion.

Figure 4.8 illustrates that negative clusters contain repeated punctuation. A significant advantage of clustering n-grams is that varying permutations of punctuation (e.g., *"?!"*

Figure 4.7: Top 5 positive concepts extracted via baseline semantic clustering ($w = 0$).

Figure 4.8: Top 5 negative concepts extracted via baseline semantic clustering ($w = 0$).

and *"!??"*) are grouped into a single conceptual feature, which theoretically could improve the model's ability to generalize noisy text. Other top negative concepts contain direct negative descriptors (e.g., *"worst"*, *"horrible"*, *"awful"*). The cluster containing the phrase *"bad, but"* appears highly ranked for negativity, even though such phrasing depicts a mitigating clause or mixed sentiment in natural language rather than pure negativity.

Weighted Approach

Figures 4.9 and 4.10 depict that incorporating lexicon-based polarity increases the Z-score values, and thus produces more cohesive and discriminative clusters. For example, the primary positive cluster expands to integrate *"excellent"* alongside *"great"*. Similarly, the *"bad"* cluster becomes more distinct and captures negative connotations, confirming the effectiveness of the weighting strategy. Descriptive but sentiment-neutral concepts (such as the *"his"* cluster) decrease in prominence, demonstrating that VADER-weighting effectively prioritizes sentiment signals.

Figure 4.9: Top 5 positive concepts extracted via weighted semantic clustering ($w = 10$).

Figure 4.10: Top 5 negative concepts extracted via weighted semantic clustering ($w = 10$).

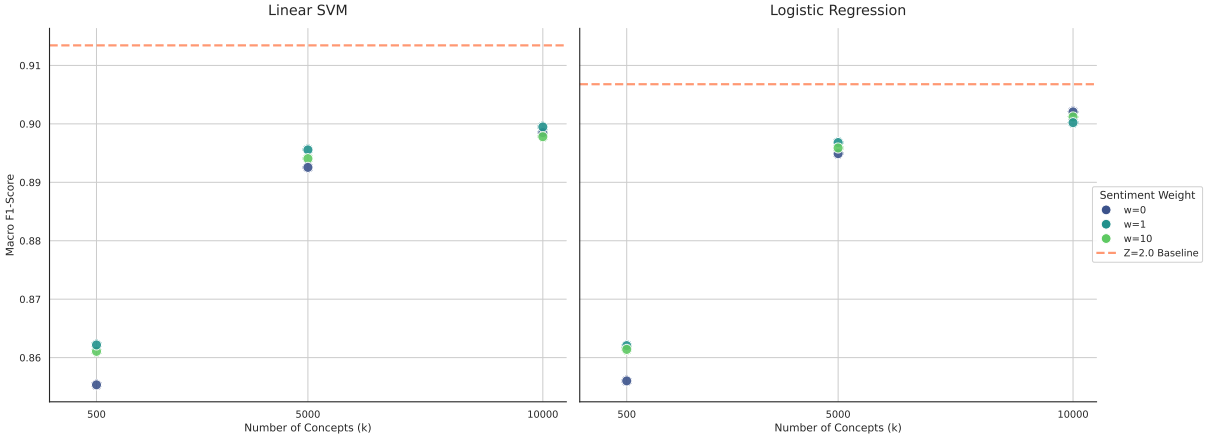


Figure 4.11: Performance comparison of clustered concepts across varying number of concepts (k) and sentiment weights (w). The left and right panels detail results for the Linear SVM and LR models, respectively. Marker colors denote the applied sentiment weight ($w \in \{0, 1, 10\}$), while the horizontal dashed line represents the unclustered $Z = 2.0$ baseline performance for each respective model.

Concept Granularity

Compressing the vocabulary to n concepts results in performance degradation for both the linear SVM and LR models. As the number of clusters increases from $k = 500$ to $k = 5\,000$ and $k = 10\,000$, the macro F_1-score improves, indicating that preserving nuances in vocabulary is required for effective sentiment analysis. This performance trajectory appears bounded by the benchmark; even at $k = 10\,000$, the clustered representations fail to surpass the unclustered baseline. Figure 4.11 highlights the benefit of sentiment weighting, especially at the lowest granularity ($k = 500$). Seemingly, injecting external sentiment knowledge mitigates information loss when the feature space is heavily compressed.

4.4.7 Impact of Certainty

In a cascade design the delegation decision depends on whether the predicted probability reflects genuine confidence. Figure 4.12 shows the probability distributions of the linear SVM and LR on the validation set. While both models concentrate predictions near the extremes, the SVM places a markedly larger share in the $[0.0, 0.1] \cup [0.9, 1.0]$ interval, whereas LR spreads predictions more evenly across intermediate bins.

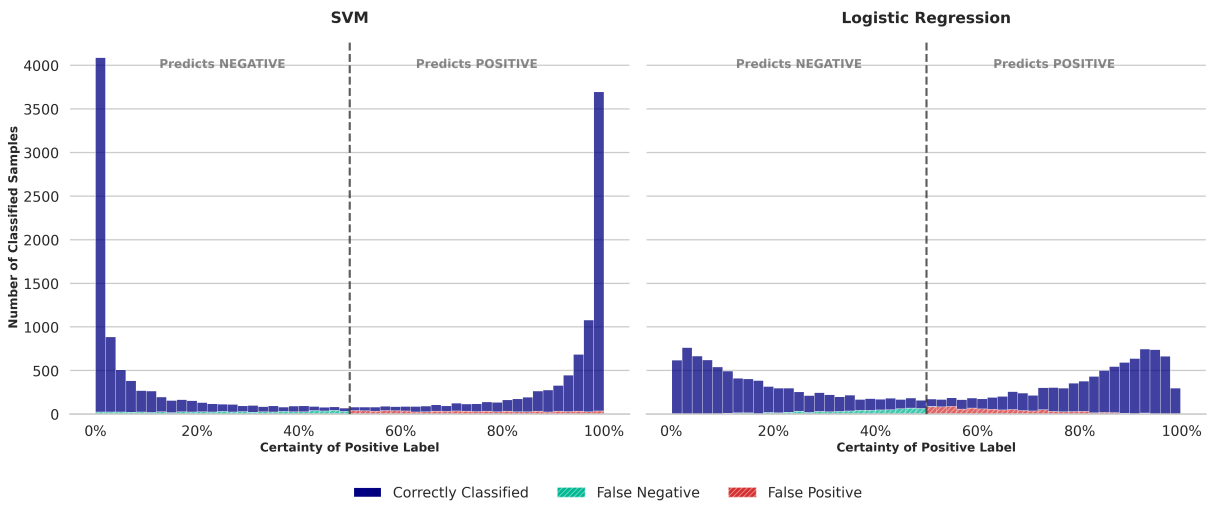


Figure 4.12: Distribution of prediction certainty for the Linear SVM (left) and LR (right) models. The vertical dashed line at 50% denotes the decision boundary separating negative and positive predictions. Bar colors indicate classification outcomes: dark blue represents correctly classified samples, teal indicates false negatives, and red indicates false positives.

Table 4.7 partitions the probability range into five symmetric zones and reports for each zone the number of predictions, the number of misclassified samples, the local accuracy, the workload (fraction of the whole validation set falling into that zone), and the percentage of all model errors that occur specifically in that zone.

- For the linear SVM, the two most certain zones alone cover $71.26\% + 11.61\% = 82.87\%$ of the data with local accuracies of 97.70% and 87.10%, respectively. The SVM's errors, however, are distributed relatively evenly across all bins; no single interval concentrates a disproportionate share of mistakes.
- LR exhibits a pronounced error concentration in the center: 42.12% of all misclassifications occur in the $[0.4, 0.6]$ band, and the error share drops sharply towards the extremes (only 2.35% in the outermost bin). This pattern is ideal for a cascade, because a narrow delegation window around the decision boundary would capture the bulk of the model's errors.

Despite this favorable error distribution, LR achieves a much smaller workload in its high-precision zone (35.72% vs. SVM's 71.26%). To match the SVM's 70% coverage, three LR bins must be combined, yielding a lower average precision. The cascade therefore can benefit more from the SVM's massive high-precision coverage, using the specialist to correct the modest, spread-out errors.

4.4.8 Impact of Cascade Ensemble

The linear SVM cascade (Figure 4.13) demonstrates superior efficiency and performance when utilizing TF-IDF representations—top 4 representations can be found in the

Table 4.7: Prediction outcomes across varying certainty ranges. The tables compare the performance of the LR and SVM models, detailing the number of predictions, local accuracy, proportional workload, and relative error contribution within each symmetric probability bracket.

(a) Certainty distribution and performance metrics for the LR model.

Probability Range	Predictions	Incorrect	Local Accuracy	Workload	Error
$[0.4, 0.6]$	1 747	681	61.02%	10.07%	42.12%
$[0.3, 0.4) \cup (0.6, 0.7]$	2 080	443	78.70%	11.99%	27.40%
$[0.2, 0.3) \cup (0.7, 0.8]$	2 803	295	89.48%	16.15%	18.24%
$[0.1, 0.2) \cup (0.8, 0.9]$	4 524	160	96.46%	26.07%	9.89%
$[0.0, 0.1) \cup (0.9, 1.0]$	6 199	38	99.39%	35.72%	2.35%

(b) Certainty distribution and performance metrics for the SVM model.

Certainty Range	Predictions	Incorrect	Local Accuracy	Workload	Error
$[0.4, 0.6]$	834	372	55.40%	4.81%	24.77%
$[0.3, 0.4) \cup (0.6, 0.7]$	929	286	69.21%	5.35%	19.04%
$[0.2, 0.3) \cup (0.7, 0.8]$	1 210	299	75.29%	6.97%	19.91%
$[0.1, 0.2) \cup (0.8, 0.9]$	2 015	260	87.10%	11.61%	17.31%
$[0.0, 0.1) \cup (0.9, 1.0]$	12 365	285	97.70%	71.26%	18.97%

legend. The SVM begins with a high initial local accuracy (above 91.0% at the 50% certainty threshold) and scales smoothly to 100.0% as the required confidence increases. Ultimately, the SVM cascade achieves a high overall combined accuracy, peaking at approximately 93.1%. Crucially, it attains this global optimization peak while delegating only 15% to 20% of the data to BERT. This visually confirms the earlier observation that the SVM provides a massive zone of high-precision coverage, allowing the system to rely heavily on the lightweight model without sacrificing accuracy.

The LR cascade (Figure 4.14), predominantly utilizing BoW representations, exhibits a higher dependency on the specialist model. The local accuracy on the retained data climbs steadily from roughly 90.0% to 100.0% as the prediction certainty threshold increases. Consequently, the deferral curve illustrates an exponential increase in the workload delegated to BERT. While the combined cascade accuracy still significantly outperforms the BERT global baseline of 91.3%, reaching an optimal peak at approximately 92.9% to 93.0%, achieving this maximum performance requires delegating roughly 25% to 30% of the most uncertain data to the specialist model.

Overall, both configurations validate the cascade methodology, proving that deferring uncertain predictions yields better accuracy than relying on the BERT baseline alone. Nevertheless, the linear SVM cascade is markedly superior to the LR cascade, as it achieves a higher maximum accuracy while delegating significantly less data to the computationally expensive specialist model.

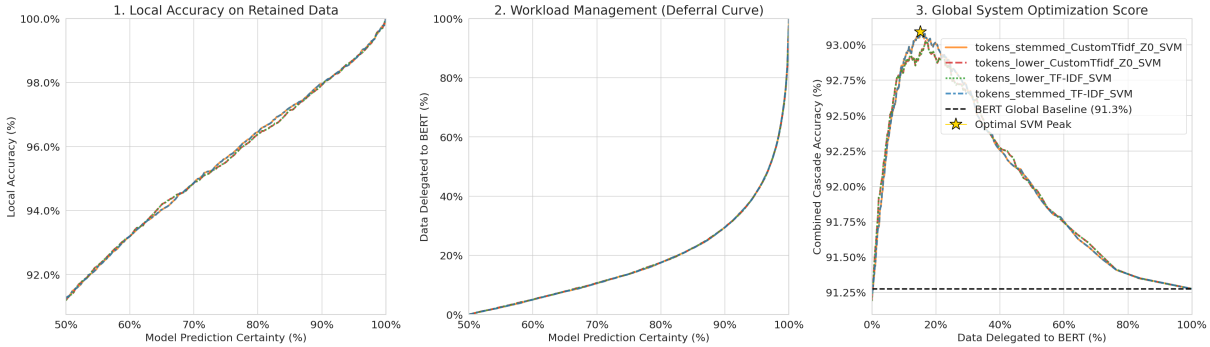


Figure 4.13: Accuracy-coverage trade-off and delegation dynamics for the SVM cascade system. The left panel illustrates the local accuracy on retained data relative to the model prediction certainty threshold. The center panel presents the deferral curve, detailing the percentage of data delegated to BERT model across varying certainty thresholds. The right panel plots the combined cascade accuracy against the delegated workload, where the horizontal dashed line indicates the BERT global baseline and the star marker highlights the optimal system configuration.

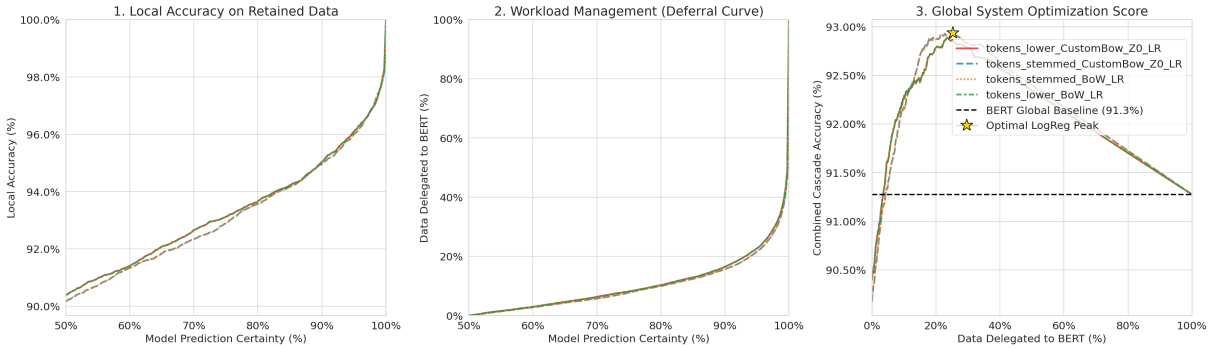


Figure 4.14: Accuracy-coverage trade-off and delegation dynamics for the LR cascade system. The left panel illustrates the local accuracy on retained data relative to the model prediction certainty threshold. The center panel presents the deferral curve, detailing the percentage of data delegated to BERT model across varying certainty thresholds. The right panel plots the combined cascade accuracy against the delegated workload, where the horizontal dashed line indicates the BERT global baseline and the star marker highlights the optimal system configuration.

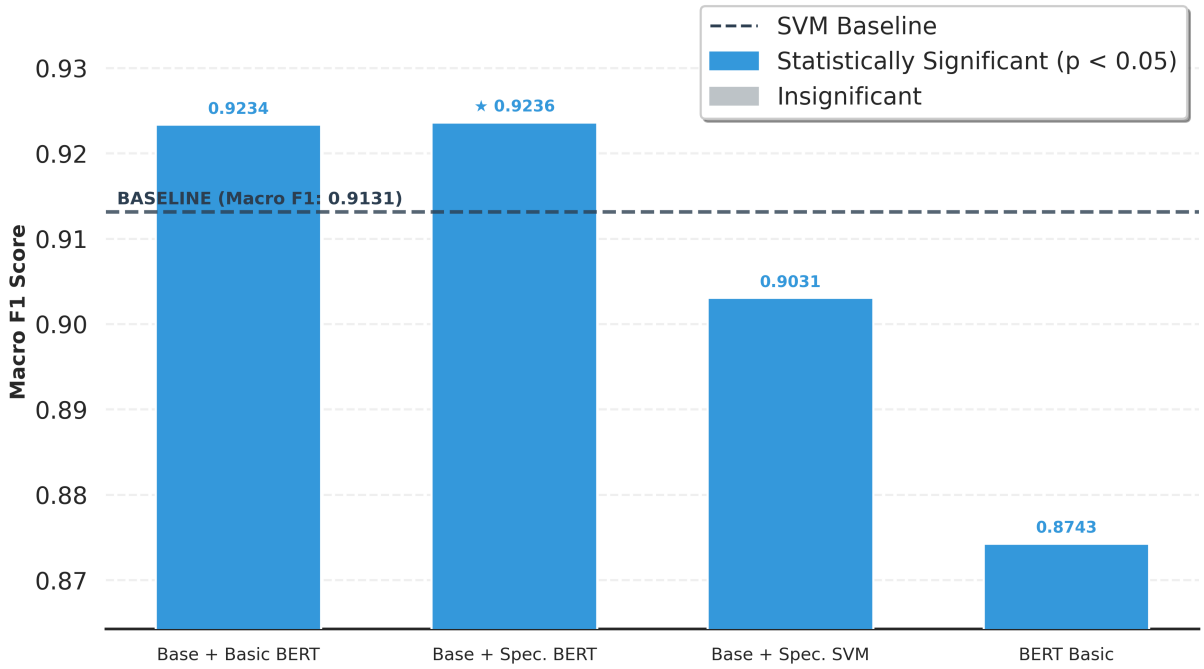


Figure 4.15: Comparison of macro F_1-scores across different cascade system configurations. The horizontal dashed line denotes the SVM baseline performance (0.9131). Bar colors indicate statistical significance relative to the baseline (blue for $p < 0.05$, grey for insignificant results), and exact scores are annotated above each bar. A star marker highlights the top-performing system configuration.

Specialist Types

As shown in Figure 4.15, introducing a specialist yields mixed results depending on its underlying mechanics—thus the choice of architecture for the specialist is crucial.

- Both BERT cascade systems successfully surpass the SVM baseline's macro F_1-score of 91.31%. Fine-tuning a BERT specialist specifically on "hard cases" mined from the validation set results in a negligible performance bump over a basic BERT model (92.36% versus 92.34%, $p = 0.859684$). This lack of statistical significance likely stems from the relatively small volume of hard samples available for fine-tuning.

- The Basic BERT model alone struggled with this specific data shuffle, achieving only an 87.43% F_1-score on the test set, compared to its validation score of 91.27%. While this difficulty explains why the cascade approach dropped by almost 1% overall, it highlights an interesting dynamic: combining two individually weaker models can still produce a more robust and improved overall score.

- Deploying a secondary SVM as the specialist model degrades the system's performance, dragging the macro F_1-score down to 0.9031. This deterioration points to a structural limitation: the base classifier has already exploited the linear feature space. Any remaining uncertain predictions rely on subtle, highly contextual cues.

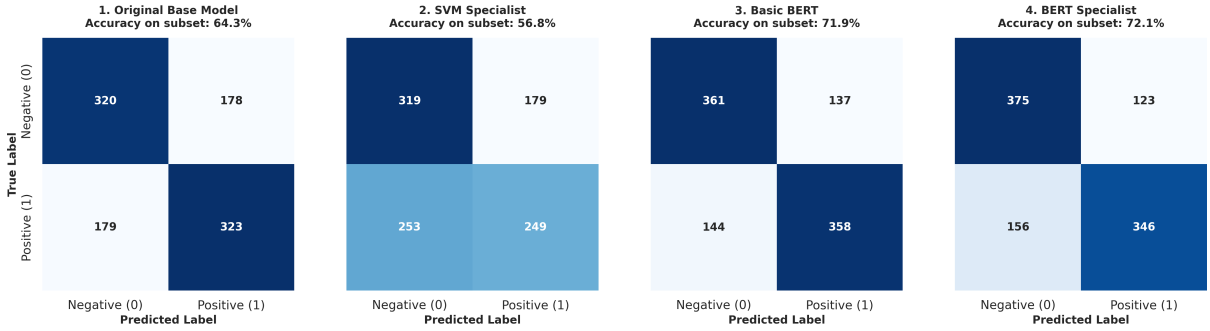


Figure 4.16: Confusion matrices evaluating model performance on a subset of hard samples. From left to right, the panels display the classification results for the Original Base Model, SVM Specialist, Basic BERT, and BERT Specialist. The intensity of the blue shading within each cell corresponds to the absolute number of samples, and the overall subset accuracy for each configuration is annotated above its respective matrix.

The Linear Limitation

This structural barrier becomes apparent when isolating model performance strictly to the most challenging samples, as illustrated by the confusion matrices in Figure 4.16.

- **Base vs. SVM:** The original base model manages a 64.3% accuracy rate on this difficult subset. When the SVM "specialist" attempts to re-evaluate these exact same cases, its accuracy actually plummets to 56.8%. This drop is primarily driven by a massive spike in false negatives, which surge from 179 to 253.
- **Semantic Capacity:** In sharp contrast, both BERT variants possess the deep semantic capacity necessary to untangle these non-linear, contextual cues. By successfully interpreting these complex patterns, the basic and specialist BERT models elevate the subset accuracy to 71.9% and 72.1%, respectively.

Chapter 5

Summary

5.1 Motivation and Objective

Sentiment analysis is a well-established NLP task, yet a persistent trade-off remains between the high accuracy of modern deep learning models and the computational efficiency required for practical deployment. This thesis investigated whether GrC techniques can bridge this gap.

5.2 Summary of Work

Granular computing approaches had not been systematically verified in the sentiment analysis pipeline. Through an enumeration of preprocessing, representation, and modeling choices, the most effective configurations were identified. The work demonstrated that a statistical Z-based pruning technique can discard the "neutral" granule—the n-grams that do not discriminate between positive and negative classes—thereby reducing the feature space while preserving or even improving accuracy. Building on this, a hybrid cascade ensemble was proposed: a lightweight linear SVM acts as a base model, handling the majority of instances with high confidence, while a deep learning specialist (DistilBERT) resolves the remaining uncertain cases. The cascade achieved a macro F_1-score of approximately 92.3%, outperforming both standalone baselines while delegating only 15%-20% of the data to the expensive model.

5.3 Conclusions

The positive aspects of the proposed approaches are lower computational cost and less sparse feature spaces. Most of the workload can be handled by the baseline SVM model, while only a small fraction of the most difficult instances is passed to the specialist. This division of labor combines efficiency with high accuracy and demonstrates that granular

principles can successfully guide the design of a practical sentiment analysis system.

5.3.1 Recommendations

- **Preprocessing is necessary, and data unification is important.** Text should be at least lowercased and expanded (contraction resolution) to ensure better generalization when using traditional machine learning approaches. Stemming and lemmatization do not significantly improve or degrade accuracy and may be omitted.

- **POS-based filtering (stop-word removal) is not advised.** Although it reduces the feature space, it imposes a ceiling on performance and is not among the best-performing representations for SVM.

- **SVMs are a much better choice than LR as baseline models.** SVMs achieve higher standalone accuracy and, critically, separate easy from difficult cases more effectively, making them more suitable for cascade architectures.

- **The linear kernel is fast and reliable.** The RBF kernel trains much longer and does not converge in an acceptable time, while offering no accuracy benefit.

- **Custom Bag-of-Words (BoW) does not benefit from pruning, but Z-based pruning works well with TF-IDF.** A pruning threshold of $Z = 2.0$ retains or even improves accuracy while discarding over 60% of the feature space.

- **There is no statistically significant difference between $(1, 2)$-grams and $(1, 3)$-grams.** Unigrams alone are not enough to capture sufficient understanding.

- **Semantic clustering works for topic classification, but is not useful for SA.** Even when external sentiment values are injected, concept-level representations erase too many polarity cues and underperform token-level features.

- **The cascade ensemble works better than expected.** Because the SVM and the transformer model operate on different feature spaces, they learn complementary aspects: the SVM handles the separation of easy from difficult cases, while the transformer focuses on understanding nuanced, contextual sentiment. The cascade achieves better results than DistilBERT or SVM baselines alone. An SVM cannot be used as a specialist because the remaining ambiguities are not resolvable within a pure vectorized (BoW or TF-IDF) representation.

- **Cascade systems can reliably use a separate validation set to establish the optimal deferral rate.** The threshold selected on held-out data generalizes well to unseen test data.

5.3.2 Research Questions Revisited

RQ1: *Can granular approaches be used to optimize the sentiment analysis pipeline?*

Yes. Statistical pruning based on Z-scores works effectively: the "not sentiment" granule is discarded, leading to a more compact and equally or more accurate feature representation.

RQ2: *Can a hybrid system be designed to combine the efficiency of lightweight classifiers with the semantic understanding of transformer models?*

Yes. A cascade ensemble can be built that improves overall performance, with the base SVM handling the majority of instances and the specialist transformer resolving only the hardest cases.

5.3.3 Further Research

Further research could verify whether higher-order n-grams (e.g., $(1, 4)$ and beyond) become feasible when combined with the proposed statistical pruning approach.

It should also be tested whether stronger models—full BERT or LLMs—could benefit from a cascade, saving inference tokens by deferring only a small fraction of instances to the heavyweight model; it is possible that such models are already so strong that the cascade would weaken their performance.

Another direction is to test whether semantic clustering could be applied at the sentence level, where similar sentences are grouped into prototypical concepts. This would require a specialized classifier that maps a sentence to its concept representative, since the set of possible sentences is unlimited and a static mapping matrix—as used here for n-grams—would not be possible.

Bibliography

[1] Benaissa Azzeddine, Azza Harbaoui, and Henda Ben Ghezala. Sentiment analysis approaches based on granularity levels. In *Proceedings of the 14th International Conference on Web Information Systems and Technologies - WEBIST*, pages 324–331, 01 2018.

[2] Stefano Baccianella, Andrea Esuli, and Fabrizio Sebastiani. Sentiwordnet 3.0: An enhanced lexical resource for sentiment analysis and opinion mining. In *Proceedings of the Seventh International Conference on Language Resources and Evaluation (LREC 2010)*, volume 10, pages 2200–2204, 01 2010.

[3] Andrzej Bargiela and Witold Pedrycz. *Granular Computing: An Introduction.* Springer, Boston, MA, 2003.

[4] S. Behdenna, F. Barigou, and G. Belalem. Document level sentiment analysis: A survey. *EAI Endorsed Transactions on Context-aware Systems and Applications*, 4(13):1–8, 3 2018.

[5] Rashid Behzadidoost, Farnaz Mahan, and Habib Izadkhah. Granular computing-based deep learning for text classification. *Information Sciences*, 652:119746, 2024.

[6] Antonino Capillo, Enrico De Santis, Fabio Massimo Frattale Mascioli, and Antonello Rizzi. Mining m-grams by a granular computing approach for text classification. In *Proceedings of the 12th International Joint Conference on Computational Intelligence (IJCCI 2020) - Volume 1: NCTA*, pages 350–360, 01 2020.

[7] Iti Chaturvedi, Erik Cambria, Roy E. Welsch, and Francisco Herrera. Distinguishing between facts and opinions for sentiment analysis: Survey and challenges. *Information Fusion*, 44:65–77, 2018.

[8] Jie Chen, Yue Chen, Yechen He, Yang Xu, Shu Zhao, and Yanping Zhang. A classified feature representation three-way decision model for sentiment analysis. *Applied Intelligence*, 52(7):7995–8007, 2022.

[9] Kyunghyun Cho, Bart Van Merriënboer, Çağlar Gulçehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations

using rnn encoder–decoder for statistical machine translation. In *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*, pages 1724–1734, 2014.

[10] Ronan Collobert and Jason Weston. A unified architecture for natural language processing: deep neural networks with multitask learning. In *Proceedings of the 25th International Conference on Machine Learning*, ICML '08, pages 160–167, New York, NY, USA, 2008. Association for Computing Machinery.

[11] Enrico De Santis and Antonello Rizzi. Prototype theory meets word embedding: A novel approach for text categorization via granular computing. *Cognitive Computation*, 15:976–997, 03 2023.

[12] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pages 4171–4186, 2019.

[13] Rehab M. Duwairi. Sentiment analysis for dialectical arabic. In *2015 6th International Conference on Information and Communication Systems (ICICS)*, pages 166–170, 2015.

[14] Hamido Fujita, Tianrui Li, and Yiyu Yao. Advances in three-way decisions and granular computing. *Knowledge-Based Systems*, 91:1–3, 2016. Three-way Decisions and Granular Computing.

[15] M. Govindarajan. Sentiment analysis of movie reviews using hybrid method of naive bayes and genetic algorithm. *International Journal of Advanced Computer Research*, 3:139–145, 2013.

[16] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.

[17] Faliang Huang, Changan Yuan, Yingzhou Bi, Jianbo Lu, Liqiong Lu, and Xing Wang. Multi-granular document-level sentiment topic analysis for online reviews. *Applied Intelligence*, 52(7):7723–7733, 05 2022.

[18] C.J. Hutto and Eric Gilbert. Vader: A parsimonious rule-based model for sentiment analysis of social media text. *Proceedings of the International AAAI Conference on Web and Social Media*, 8:216–225, 05 2014.

[19] Md Shofiqul Islam, Muhammad Nomani Kabir, Ngahzaifa Ab Ghani, Kamal Zuhairi Zamli, Nor Saradatul Akmar Zulkifli, Md Mustafizur Rahman, and Mohammad Ali

Moni. Challenges and future in deep learning for sentiment analysis: a comprehensive review and a proposed novel hybrid approach. *Artificial Intelligence Review*, 57(3):62, 2024.

[20] Liping Jing and Raymond Y. K. Lau. Granular computing for text mining: New research challenges and opportunities. In *Rough Sets, Fuzzy Sets, Data Mining and Granular Computing*, volume 5908 LNAI of *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, pages 478–485. Springer Verlag, 2009. 12th International Conference on Rough Sets, Fuzzy Sets, Data Mining and Granular Computing, RSFDGrC 2009 ; Conference date: 15-12-2009 Through 18-12-2009.

[21] Archa Joshy and Sumod Sundar. Analyzing the performance of sentiment analysis using bert, distilbert, and roberta. In *2022 IEEE International Power and Renewable Energy Conference (IPRECON)*, pages 1–6, 2022.

[22] Haihui Li, Ting Yuan, Haiming Wu, Yun Xue, and Xiaohui Hu. Granular computing-based multi-level interactive attention networks for targeted sentiment analysis. *Granular Computing*, 5(3):387–395, 07 2020.

[23] Hang Li. Deep learning for natural language processing: advantages and challenges. *National Science Review*, 5(1):24–26, 09 2017.

[24] Bing Liu. *Sentiment Analysis: A Fascinating Problem*, pages 1–8. Springer International Publishing, Cham, 2012.

[25] Han Liu and Mihaela Cocea. Fuzzy information granulation towards interpretable sentiment analysis. *Granular Computing*, 2(4):289–302, 04 2017.

[26] Andrew L. Maas, Raymond E. Daly, Peter T. Pham, Dan Huang, Andrew Y. Ng, and Christopher Potts. Learning word vectors for sentiment analysis. In *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies*, pages 142–150, Portland, Oregon, USA, 06 2011. Association for Computational Linguistics.

[27] Malik Muhammad Saad Missen, Mohand Boughanem, and Guillaume Cabanac. Opinion mining: Reviewed from word to document level. *Social Network Analysis and Mining*, 3:107–125, 03 2012.

[28] Restu Mulianingrum and Erwin Yudi Hidayat. Comparative performance of SVM and BERT-base using hybrid preprocessing for fast fashion sentiment analysis. *Journal of Applied Informatics and Computing*, 9:3464–3478, 12 2025.

[29] Lunyiu Nie, Zhimin Ding, Erdong Hu, Christopher Jermaine, and Swarat Chaudhuri. Online cascade learning for efficient inference over streams, 2024.

[30] Ahmad Nursal. Battle of sentiment lexicons: Wordnet, Sentiwordnet, Textblob and Vader in Web Forum Analysis. *Journal of Information Systems Engineering and Management*, 10:84–93, 01 2025.

[31] Alexander Pak and Patrick Paroubek. Classification en polarité de sentiments avec une représentation textuelle à base de sous-graphes d'arbres de dépendances (sentiment polarity classification using a textual representation based on subgraphs of dependency trees). In *JEPTALNRECITAL*, 2011.

[32] Bo Pang, Lillian Lee, and Shivakumar Vaithyanathan. Thumbs up? sentiment classification using machine learning techniques. In *Proceedings of the 2002 conference on empirical methods in natural language processing (EMNLP 2002)*, pages 79–86, 2002.

[33] Mario Paredes-Valverde, Ricardo Colomo-Palacios, María Salas Zarate, and Rafael Valencia-García. Sentiment analysis in Spanish for improvement of products and services: A deep learning approach. *Scientific Programming*, 2017:1–6, 10 2017.

[34] Jeffrey Pennington, Richard Socher, and Christopher Manning. Glove: Global vectors for word representation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, volume 14, pages 1532–1543, 01 2014.

[35] Francesca Possemato and Antonello Rizzi. Automatic text categorization by a granular computing approach: Facing unbalanced data sets. *The 2013 International Joint Conference on Neural Networks (IJCNN)*, pages 1–8, 2013.

[36] Taorong Qiu, Xiaoqing Chen, Qing Liu, and Houkuan Huang. Granular computing based text classification. In *2006 IEEE International Conference on Granular Computing*, pages 313–316, 2006.

[37] Kumar Ravi. A survey on opinion mining and sentiment analysis: Tasks, approaches and applications. *Knowledge-Based Systems*, 89:14–46, 06 2015.

[38] Mohammed Rushdi-Saleh, M. Teresa Martín-Valdivia, L. Alfonso Ureña-López, and José M. Perea-Ortega. Oca: Opinion corpus for Arabic. *Journal of the American Society for Information Science and Technology*, 62(10):2045–2054, 2011.

[39] Nirmal Adhikari Sharma, A. B. M. Shawkat Ali, and Muhammad Ashad Kabir. A review of sentiment analysis: tasks, applications, and deep learning techniques. *International Journal of Data Science and Analytics*, 19(3):351–388, 2025.

[40] Carlo Strapparava and Alessandro Valitutti. Wordnet-affect: an affective extension of wordnet. *Vol 4.*, 4:1083–1086, 01 2004.

[41] Emma Strubell, Ananya Ganesh, and Andrew McCallum. Energy and policy considerations for deep learning in nlp. In *Proceedings of the 57th annual meeting of the association for computational linguistics*, pages 3645–3650, 2019.

[42] Duyu Tang and Ting Liu. Document modeling with gated recurrent neural network for sentiment classification. In *Proceedings of the 2015 conference on empirical methods in natural language processing*, pages 1422–1432, 01 2015.

[43] Qurat Tul, Mubashir Ali, Amna Riaz, Amna Noureen, Muhammad Kamranz, Babar Hayat, and Aziz Rehman. Sentiment analysis using deep learning techniques: A review. *International Journal of Advanced Computer Science and Applications*, 8:424–433, 01 2017.

[44] Neeraj Varshney and Chitta Baral. Model cascading: Towards jointly improving efficiency and accuracy of NLP systems, 2022.

[45] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *31st Conference on Neural Information Processing Systems (NIPS)*, pages 1–15, 06 2017.

[46] G Vinodhini and RM Chandrasekaran. Effect of feature reduction in sentiment analysis of online reviews. *Int J Adv Res Comput Eng Technol (IJARCET)*, 2(6):2165–2172, 2013.

[47] Ashima Yadav and Dinesh Kumar Vishwakarma. Sentiment analysis using deep learning architectures: a review. *Artificial intelligence review*, 53(6):4335–4385, 2020.

[48] Xin Yang, Yujie Li, Qiuke Li, Dun Liu, and Tianrui Li. Temporal-spatial three-way granular computing for dynamic text sentiment classification. *Information Sciences*, 596:551–566, 2022.

[49] Xin Yang, Yujie Li, Dan Meng, Yuxuan Yang, Dun Liu, and Tianrui Li. Three-way multi-granularity learning towards open topic classification. *Information Sciences*, 585:41–57, 2022.

[50] Yiyu Yao. Three-way decision and granular computing. *International Journal of Approximate Reasoning*, 103:107–123, 2018.

[51] Lotfi A. Zadeh. Toward a theory of fuzzy information granulation and its centrality in human reasoning and fuzzy logic. *Fuzzy Sets and Systems*, 90(2):111–127, 1997. Fuzzy Sets: Where Do We Stand? Where Do We Go?

[52] Lin Zhang, Kun Hua, Honggang Wang, Guanqun Qian, and Li Zhang. Sentiment analysis on reviews of mobile users. *Procedia Computer Science*, 34:458–465, 2014. The 9th International Conference on Future Networks and Communications (FNC'14)/The 11th International Conference on Mobile Systems and Pervasive Computing (MobiSPC'14)/Affiliated Workshops.

Appendices

List of abbreviations and symbols

ABSA Aspect-Based Sentiment Analysis

ANEW Affective Norms for English Words

BERT Bidirectional Encoder Representations from Transformers

BoC Bag-of-Concepts

BoW Bag-of-Words

CBoW Custom Bag-of-Words

CNN Convolutional Neural Network

CTf-IDF Custom TF-IDF

DBSCAN Density-Based Spatial Clustering of Applications with Noise

DLSA Document-Level Sentiment Analysis

ED Emotion Detection

GA Genetic Algorithm

GI General Inquirer

GNN Graph Neural Network

GPU Graphics Processing Unit

GrC Granular Computing

GRU Gated Recurrent Unit

IG Information Granulation

IMDb Internet Movie Database

LDA Latent Dirichlet Allocation

LIWC Linguistic Inquiry and Word Count

LLM Large Language Models

LR Logistic Regression

LSTM Long Short-Term Memory

MMSA Multimodal Sentiment Analysis

NB Naïve Bayes

NBSVM Naïve Bayes SVM

NER Named Entity Recognizer

NLP Natural Language Processing

NLTK Natural Language Toolkit

OCA Opinion Corpus for Arabic

OSD Opinion Spam Detection

PCA Principal Component Analysis

PMI Pointwise Mutual Information.

POS Part-Of-Speech

RBF Radial Basis Function

RNN Recurrent Naural Network

RQ# Research Question #

SA Sentiment Analysis

SLSA Sentence-Level Sentiment Analysis

SO Semantic Orientation

SVM Support Vector Machine

TAO Trisecting-Acting-Outcome

TF-IDF Term Frequency-Inverse Document Frequency

TPU Tensor Processing Unit

VADER Valance Aware Dictionary sEntiment Reasoner

3WD Three-Way Decision

List of Figures

List of Tables